From Source to Execution: Design and Implementation of a Statically-Typed Programming Language with Type Inference

by

Aye Khin Khin Hpone (Yolanda Lim), st125970

Applegate T. Tun Oo, st126690

Win Htut Naing, st126687

AT70.07 — Programming Languages and Compilers
Assignment 2

Instructor: Prof. Phan Minh Dung
Teaching Assistant: Akraradet Sinsamersuk

Asian Institute of Technology
School of Engineering and Technology
Thailand

April 2026

ABSTRACT

This report presents the design and implementation of a small, statically-typed interpreted programming language realised through a four-stage compiler pipeline. The language provides four primitive types — Integer, Float, Boolean, and String — and supports arithmetic and equality expressions, variable assignment with type inference from first use, conditional and iterative control flow, user-defined functions with value parameter passing, and a built-in `print()` function. Types are inferred from program context rather than declared explicitly, combining static type safety with a declaration-free syntax. The pipeline consists of a hand-written lexer, a recursive-descent parser that constructs an abstract syntax tree, a static type checker that performs inference and rejects ill-typed programs before execution, and a tree-walking interpreter. Both a command-line runner and an interactive desktop interface expose each pipeline stage for inspection. The report documents the formal grammar, the type inference algorithm, the system architecture, and the automated test suite of 54 tests that validates the complete implementation.

CONTENTS

CHAPTER 1

INTRODUCTION

Programming languages define the formal contract between a programmer's intent and machine execution. Building a language processor from source text through to evaluated output requires a coherent pipeline of components, each of which must correctly transform its input before passing it downstream. Lexical analysis identifies atomic tokens; parsing imposes hierarchical structure; static type checking validates semantic correctness before execution; and interpretation evaluates the validated program. When these stages are connected cleanly, errors are caught at the earliest possible point, and the overall system remains tractable to reason about, test, and extend.

This report presents the design and implementation of a small statically-typed interpreted language that realises this pipeline in full. The language supports four primitive types — Integer, Float, Boolean, and String — and provides arithmetic and equality expressions, assignment, conditional and iterative control flow, user-defined functions with value parameter passing, and a built-in `print()` statement. Types are inferred from program context rather than declared explicitly, combining the safety of static typing with the conciseness of a declaration-free syntax. The implementation spans three principal components developed in pipeline order: the lexer and symbol table, the recursive-descent parser and abstract syntax tree, and the type checker with interpreter. Each component was completed before the next was begun, since each stage consumes the structured output of its predecessor.

The implemented system can be invoked from the command line or through an interactive desktop interface, both of which surface the four pipeline stages — tokens, AST, inferred type table, and execution output — for inspection. An automated regression suite of 54 tests verifies correctness across all stages and language features.

The remainder of this report is organised as follows. Chapter 2 presents the language design, token specification, grammar, and operator precedence rules. Chapters 3 and 4 cover the front-end implementation in pipeline order: lexical analysis and the symbol table, then recursive-descent parsing and the AST. Chapter 5 explains the type inference algorithm and the semantic rules used by the checker. Chapter 6 presents the system architecture and implementation, including the end-to-end pipeline, project layout, and technology stack. Chapter 7 discusses testing and verification, and Chapter 8 concludes the report.

CHAPTER 2

LANGUAGE DESIGN AND GRAMMAR

2.1 Types

The language provides four primitive types, chosen to cover the core domains of numeric computation, logical control flow, and text manipulation while keeping the type system tractable and the semantics well-defined.

2.1.1 Integer

The Integer type represents whole numbers such as `0`, `7`, and `42`. Integer literals are written as one or more decimal digits without quotation marks or a decimal point.

2.1.2 Float

The Float type represents real-valued numbers written with a decimal point, such as `3.14` or `20.5`. Float values are used in arithmetic expressions and also arise as the result of any division. Division always produces a `Float` even when both operands are `Integer`: for example, `10 / 2` evaluates to `5.0 : Float`. This is an explicit design rule enforced in the type checker.

2.1.3 Boolean

The Boolean type represents logical truth values. The two Boolean literals are `true` and `false`. Boolean values are used in conditions and as results of comparison expressions.

2.1.4 String

The String type represents sequences of characters enclosed in double quotes, for example `"hello"`. Strings can be assigned to variables and printed through the built-in `print()` function. The surrounding quotes are stripped by the parser when the `Literal` node is created, so the value stored internally is the bare character sequence without surrounding quotes. At runtime, the built-in `print()` function re-adds double quotes around string values, so `print("plc")` outputs `"plc" : String`.

2.1.5 *Void*

Although the language exposes exactly four programmer-visible types, the implementation introduces a fifth value, `Void`, within the `LanguageType` enumeration. Its sole purpose is to provide a well-defined return type for functions that contain no `return` statement, allowing the symbol table to record a consistent entry for every function regardless of whether it produces a value. `Void` cannot appear in source code and is never visible to the programmer; it is a bookkeeping device used internally by the type checker.

2.2 Static Typing

The language employs static typing, meaning that type errors are detected at type-checking time rather than at runtime. No explicit variable type declarations are required. Instead, the type checker infers types from literals, assignments, operations, function calls, and return statements. Once a variable type is established, later assignments must remain consistent with that type. For example, the following program is rejected at the type-checking stage before any execution occurs:

```
x = 5;
x = "hello";    (* TypeError: Cannot assign String to variable 'x' of
    type Integer *)
```

This contrasts with dynamically-typed languages, where the same program would run without error until (or unless) a type-dependent operation was attempted.

2.3 Token Specification

Table 2.1 summarises the major token classes used by the language.

Table 2.1. Token Specification

Token Category	Example Lexemes
`INT_LITERAL`	`0`, `10`, `42`
`FLOAT_LITERAL`	`3.14`, `20.5`
`BOOL_LITERAL`	`true`, `false`
`STRING_LITERAL`	`"hello"`
`IDENTIFIER`	variable and function names such as `x`, `add`
Arithmetic operators	`+`, `-`, `*`, `/`
Comparison operators	`==`, `!=`
Assignment operator	`=`
Keywords	`if`, `else`, `while`, `def`, `return`, `print`
Delimiters	`( ) { } , ;`

2.4 Context-Free Grammar

The language grammar can be described by the context-free grammar $G = (V_N, V_T, P, S)$, where V_N is the set of non-terminals, V_T is the set of terminals, P is the set of production rules, and S is the start symbol `program`. The grammar is implemented through recursive-descent parsing functions.

2.4.1 Program Structure

```
program    -> statement* EOF
statement  -> assignment
           | if_stmt
           | while_stmt
           | func_def
           | print_stmt
           | return_stmt
           | expr ';'
```

2.4.2 Arithmetic Expressions

```
expr    -> term (('+' | '-') term)*
term    -> factor (('*' | '/') factor)*
factor  -> '-' factor
      | INT_LITERAL
      | FLOAT_LITERAL
      | STRING_LITERAL
      | BOOL_LITERAL
      | IDENTIFIER
      | IDENTIFIER '(' args? ')'
      | '(' expr ')'
```

The unary minus production allows negative literals such as `-5` and negated variable references such as `-x`. It is implemented as a right-recursive descent into `factor` and is desugared internally to `0 - factor`, so all existing type rules for arithmetic apply without modification.

2.4.3 Boolean Expressions

```
bool_expr -> expr '==' expr
           | expr '!=' expr
           | BOOL_LITERAL
           | expr           (* fallback: type checker enforces Boolean *)
```

Boolean expressions are only parsed as conditions inside `if` and `while` statements. Comparison results are not storable in variables directly; the grammar does not include `bool_expr` in the `factor` rule. This is consistent with the assignment specification, which describes Boolean expressions primarily as conditions.

If `bool_expr` parses an expression and finds no following == or `!=` operator, the expression is returned as-is and the type checker verifies that its inferred type is `Boolean`.

2.4.4 *Statements*

```
assignment  -> IDENTIFIER '=' expr ';'
if_stmt     -> 'if' '(' bool_expr ')' block ('else' block)?
while_stmt  -> 'while' '(' bool_expr ')' block
print_stmt  -> 'print' '(' expr ')' ';'
return_stmt -> 'return' expr ';'
block       -> '{' statement* '}'
```

2.4.5 *Function Definitions and Calls*

```
func_def -> 'def' IDENTIFIER '(' params? ')' block
params   -> IDENTIFIER (',' IDENTIFIER)*
args     -> expr (',' expr)*
```

Function calls use value parameter passing. This means the argument values are evaluated before the call and copied into the function's local environment.

2.5 Operator Precedence and Associativity

Operator precedence is encoded structurally in the recursive-descent parser. Comparison is lowest, addition and subtraction are next, and multiplication and division have the highest precedence among binary operators. Arithmetic operators are left-associative.

Table 2.2. Operator Precedence (1 = lowest, 3 = highest) and Associativity

Precedence	Operators	Associativity
1 (lowest)	==, !=	non-associative (condition use only)
2	+, -	left-associative
3 (highest)	*, /	left-associative

CHAPTER 3

LEXICAL ANALYSIS AND SYMBOL TABLE

This chapter covers the responsibilities related to lexical and early semantic foundations of the language. It includes defining token structures and categories (such as keywords, operators, identifiers, and literals), implementing the lexer to transform source characters into tokens, and establishing the symbol table to manage variable and function storage along with their associated types. Together, these components ensure that raw source input is systematically organised into meaningful structures that can be used reliably by later stages of the compiler.

3.1 Token representation

The token system, written in `components/tokens.py`, establishes the vocabulary and structure of all lexical elements in the language. It specifies both the types of tokens that can exist and the structure each token must follow. This serves as a canonical contract between the lexer and the parser, ensuring consistency across all stages of the compiler.

Table 3.1 summarises the main categories, example lexemes, and typical `TokenType` values.

Table 3.1. Token categories and representative `TokenType` values

Category	Example lexemes	Representative `TokenType`
Literals	`42`, `3.14`, `true`, `"hello"`	`INT_LITERAL`, `FLOAT_LITERAL`, `BOOL_LITERAL`, `STRING_LITERAL`
Identifiers	`x`, `counter`, `add`	`IDENTIFIER`
Keywords	`if`, `else`, `while`, `def`, `return`, `print`	keyword-specific token variants
Operators	`+`, `-`, `*`, `/`, `=`, `==`, `!=`	`PLUS`, `MINUS`, `STAR`, `SLASH`, `ASSIGN`, `EQUAL_EQUAL`, `BANG_EQUAL`
Delimiters	Parentheses, braces, comma, and semicolon (see lexer delimiter tokens)	delimiter token variants
End marker	end of source input	`EOF`

Token types (from class `TokenType`) are defined using an `Enum` with `auto()` to automatically assign unique values. This avoids reliance on raw strings and ensures safer comparisons during parsing. The token categories include literals, identifiers, keywords, operators, delimiters, and a special end-of-file (`EOF`) token. The `EOF` token is appended after all input is processed, allowing the parser to detect the end of input explicitly rather than encountering an unexpected termination.

Each token is represented as an immutable dataclass (`frozen=True`), meaning its fields cannot be modified after creation. Each token contains four key pieces of information: its kind (`token_type`), its lexeme (the exact substring from the source code), and its source position (`line`, `column`). This structured representation allows later stages to process the program in terms of syntax and errors without handling raw text directly. Immutability improves reliability by preventing accidental modification during later processing.

```
@dataclass(frozen=True)
class Token:
    token_type: TokenType
    lexeme: str
    line: int
    column: int
```

3.2 Lexer implementation

The lexer, implemented in `components/lexica.py`, is a hand-written scanner that converts raw source text into a sequence of tokens. Unlike generated lexers (for example SLY lexer mode, or Lex and Flex), this implementation uses explicit control flow (`if` and `while`) to process input.

The lexer maintains internal state including the source string, current position, start of the current token, line number, and column index. The column index is tracked per line and resets upon encountering a newline, while a separate `token_column` records the starting position of each token for accurate error reporting.

The main function, `tokenize`, iterates through the source code, repeatedly invoking `_scan_token` to identify the next token. Tokens are appended to a list, and an `EOF` token is added at the end. Whitespace is skipped, while invalid input results in a `LexerError`.

```
def tokenize(self) -> list[Token]:
    tokens: list[Token] = []
    while not self._is_at_end():
        self.start = self.current
        self.token_column = self.column
        token = self._scan_token()
        if token is not None:
            tokens.append(token)
    tokens.append(Token(TokenType.EOF, "", self.line, self.column))
    return tokens
```

Token recognition follows a structured approach. Single-character tokens are mapped using a lookup table. Multi-character operators such as `==` and `!=` are handled using lookahead via `_match`. String literals are processed by `_string`, which reads characters until a closing quotation mark or the end of input is reached, with error handling for unterminated strings. Numeric literals are handled by `_number`, which distinguishes integers and floating-point values based on the presence of a decimal point followed by digits. Identifiers are processed greedily, consuming alphanumeric characters and underscores, and then checked against the `KEYWORDS` map to determine whether they are reserved words or user-defined names. Notably, `true` and `false` are also entries in this map, mapping to `TokenType.BOOL_LITERAL` rather than a keyword-specific type. This means boolean literals are recognised through the same keyword-fallback mechanism as reserved words, preventing them from being scanned as user-defined identifiers.

```
KEYWORDS: dict[str, TokenType] = {
    "if": TokenType.IF,
    "else": TokenType.ELSE,
    "while": TokenType.WHILE,
    # ...
    "true": TokenType.BOOL_LITERAL,
    "false": TokenType.BOOL_LITERAL,
}
```

```
SINGLE_CHAR_TOKENS: dict[str, TokenType] = {
    "+": TokenType.PLUS,
    "-": TokenType.MINUS,
    "*": TokenType.STAR,
    "/": TokenType.SLASH,
    # ...
}
```

Helper functions such as `_advance`, `_peek`, and `_peek_next` manage character consumption and lookahead, while `_is_at_end` determines when input has been fully processed. Error messages include precise line and column information, aiding debugging and diagnostics.

Table 3.2 lists the main functions and data structures in `components/lexica.py` and their roles.

Table 3.2. Key functions and data in `lexica.py`

Function / data	Responsibility	Output / effect
`tokenize()`	Iterates through source, scans tokens, appends `EOF`	ordered token list
`_scan_token()`	Dispatches recognition based on character and lookahead	next token or skip
`_number()`	Parses numeric sequences, distinguishes int vs float	numeric literal token
`_string()`	Parses quoted text, checks termination	string token or error
`_identifier()`	Parses identifiers, resolves keyword vs user-defined	keyword or identifier token
`KEYWORDS` map	Maps reserved words to token types	token type selection
`SINGLE_CHAR_TOKENS` map	Maps single-character symbols	token type selection
`_advance()`, `_peek()`, `_match()`	Character consumption and lookahead	scanner state update

3.3 Symbol table and semantic structure

The symbol table, defined in `components/symbol_table.py`, operates at a higher level than the lexer. While the lexer processes raw text into tokens, the symbol table manages semantic information such as variable and function names, their types, and their scope.

The symbol table is implemented as a scoped structure, where each scope maintains its own dictionary of symbols and optionally references a parent scope. This enables proper handling of nested blocks and variable shadowing. Symbols are introduced through definition functions and retrieved through lookup operations, which search recursively through parent scopes if needed.

```python
def lookup(self, name: str) -> Symbol:
    symbol = self._symbols.get(name)
    if symbol is not None:
        return symbol
    if self.parent is not None:
        return self.parent.lookup(name)
    raise SymbolTableError(f"Undefined symbol: {name}")
```

The type system is represented using a `LanguageType` enumeration, which includes fundamental types such as Integer, Float, Boolean, String, and Void. Symbols are modelled using an inheritance hierarchy: the base `Symbol` class stores a name and type, while subclasses such as `VariableSymbol` and `FunctionSymbol` include additional attributes. Variables track initialization state, while functions store parameter lists and return types. Table 3.3 summarises the main building blocks.

Table 3.3. Symbol table components and roles

Component	Stored information	Role in semantic analysis
Scoped symbol table	Symbols in current and parent scope	Supports nesting and shadowing via recursive lookup
`LanguageType` enum	Integer, Float, Boolean, String, Void	Defines the type system
`Symbol` (base)	Name and declared or inferred type	Base abstraction for entries
`VariableSymbol`	Variable metadata (including initialization state)	Tracks correct usage before assignment
`FunctionSymbol`	Parameters and return type	Validates calls and return consistency
`lookup(name)`	Recursive scope search	Resolves identifiers or reports undefined
`SymbolTableError`	Semantic error details	Reports duplicates, undefined symbols, invalid operations

Errors related to semantic violations are handled via `SymbolTableError`. These include duplicate declarations within the same scope, references to undefined symbols, and invalid operations such as attempting to mark a non-variable symbol as initialized.

3.4 Role of the symbol table in the pipeline

The symbol table is constructed during the type-checking phase rather than during lexical analysis. This guideline is followed in the implementation. The type checker initialises a global symbol table and populates it while traversing the AST, including defining variables and functions, managing nested scopes, and validating type correctness.

```
def run_pipeline(source_code: str) -> PipelineResult:
    tokens = Lexer(source_code).tokenize()
    tree = Parser(tokens).parse()
    ast_text = ASTPrinter().print(tree)
    checked_scope = TypeChecker().check(tree)
    outputs: list[str] = []
    Interpreter(output_callback=outputs.append).execute(tree)
    return PipelineResult(
        tokens=tokens,
        ast_text=ast_text,
        checked_scope=checked_scope,
        outputs=outputs,
    )
```

Once type checking is complete, the populated symbol table is returned as part of the pipeline result (`checked_scope`). The checker creates child scopes for blocks and functions during analysis, but the displayed `checked_scope` table represents the global scope entries returned by the pipeline. It can then be displayed or used for further analysis.

This separation of concerns keeps lexical analysis focused on tokenization, while semantic meaning is enforced during type checking. For user-defined functions, definitions are first registered in the global scope with placeholder parameter and return types. These are inferred from the first valid call and refined through body checking; subsequent calls must match the inferred parameter types.

CHAPTER 4

RECURSIVE-DESCENT PARSING AND THE ABSTRACT SYNTAX TREE

This chapter presents the implementation of the recursive-descent parser and the associated abstract syntax tree (AST) model. Starting from lexer-produced tokens, the parser constructs a hierarchical `Program` AST that represents expressions, statements, and control-flow structure in a form suitable for downstream semantic analysis and execution. Implemented syntax coverage includes arithmetic expressions, assignments, if-then-else, while, function definitions, function calls, return, and built-in `print(...)` statements. Equality operators (`==`, `!=`) are parsed in condition contexts (if/while) through the parser's boolean-expression path.

4.1 Abstract syntax tree representation

The AST is defined as a collection of Python dataclasses representing the structural elements of the programming language. It serves as an intermediate representation between parsing and later stages such as type checking and interpretation.

A key distinction exists between tokens and AST nodes. Tokens are flat and describe individual lexical elements such as operators or identifiers. In contrast, the AST is hierarchical. For example, while a token may represent the symbol `+`, a `BinaryOp` node represents an entire addition operation, including its left and right operands as subtrees.

All AST nodes inherit from a common base class, allowing them to store positional metadata (`line`, `column`). This ensures that errors detected in later stages can still reference precise locations in the original source code.

```python
@dataclass
class ASTNode:
    """Base class for every AST node. Carries source location for
        diagnostics."""
    line: int = field(default=0, kw_only=True)
    column: int = field(default=0, kw_only=True)
```

4.1.1 Expressions

Expressions represent computations or values within the program. A `Literal` node stores values known at parse time, including integers, floating-point numbers, booleans, and strings. String literals are stored without surrounding quotation marks, as these are removed during parsing.

```
if tok.token_type == TokenType.STRING_LITERAL:
    self._advance()
    return Literal(value=tok.lexeme[1:-1], line=tok.line, column=tok.
        column)
```

An `Identifier` node represents the use of a variable name within an expression, indicating evaluation rather than declaration. A `BinaryOp` node models arithmetic and comparison operations such as addition, subtraction, multiplication, division, and equality checks, storing the operator along with references to left and right operands. A `FunctionCall` node represents function invocation, consisting of the function name and a list of argument expressions.

```
@dataclass
class BinaryOp(ASTNode):
    """An arithmetic or comparison binary operation.

    op is one of: '+' '-' '*' '/' '==' '!='
    Example:  a + b   x == 0
    """
    op: str
    left: ASTNode
    right: ASTNode
```

4.1.2 Statements

Statements represent actions or control flow within the program. An `Assign` node models variable assignment, pairing a variable name with a value expression, while `Print` and `Return` nodes each store a single expression and represent output and function return operations respectively. A `Block` node groups multiple statements within braces, representing a scoped execution sequence.

Control flow is handled via `If` and `While` nodes. The `If` node stores a condition, a then-block, and an optional else-block, while `While` stores a loop condition and body block. Function definitions are represented by `FunctionDef` nodes, which include the function name, a list of parameter names, and a body block. Parameter types are intentionally not assigned at parse time, as type inference is handled later by the type checker. Finally, `Program` serves as the

AST root, containing all top-level statements.

4.2 Parser implementation

The parser consumes the list of tokens generated by the lexer and produces a single `Program` AST node. It is implemented as a hand-written recursive-descent parser based on the project grammar. This avoids parser generators and instead relies on explicit functions for each grammar rule.

The parser maintains internal state consisting of the token list and a position index (`self._pos`) indicating the current token. Tokens are accessed via helper methods such as `_peek` and `_advance`. Parsing errors are reported using a custom `ParseError` exception with line and column information consistent with lexer diagnostics.

```
def parse(self) -> Program:
    """Parse the full token stream and return the root Program node."""
    line, col = self._peek().line, self._peek().column
    statements: list[ASTNode] = []
    while not self._is_at_end():
        statements.append(self._statement())
    return Program(statements=statements, line=line, column=col)
```

In addition to the main `parse()` entry point, the parser's statement dispatcher (`_statement`) implements a deterministic decision order over token patterns: function definition, conditional, loop, return, print, assignment, then expression statement fallback. This design is important because it prevents ambiguity between syntactically similar forms. In particular, assignments are recognized using one-token lookahead (`IDENTIFIER` followed by `=`), while identifier-led expressions that are not assignments are parsed as expression statements and must still end with a semicolon. This gives the parser full coverage of both declarative-style and expression-driven top-level statements without requiring a parser generator.

4.2.1 Statement parsing

The main parsing loop repeatedly processes statements until `EOF`, with the statement type determined via dispatch on the current token. Function definitions, conditionals, loops, returns, and prints each have dedicated parsing methods. Assignments are detected using one-token lookahead (`IDENTIFIER` followed by `ASSIGN`), preventing misclassification of function calls as assignments. If no specific statement pattern matches, the parser falls back to an expression statement and enforces a terminating semicolon.

```
# assignment: IDENTIFIER "=" expr ";"
if (
    tok.token_type == TokenType.IDENTIFIER
    and self._peek_next().token_type == TokenType.ASSIGN
):
    return self._assignment()
# fallback: bare expression statement  expr ";"
node = self._expr()
self._expect(TokenType.SEMICOLON, "Expected ';' after expression")
```

4.2.2 Expression parsing and precedence

Expressions are parsed in layered precedence levels: `_expr` handles `+` and `-`, `_term` handles `*` and `/`, and `_factor` handles literals, identifiers and function calls, and parenthesized expressions. Boolean conditions for `if` and `while` are parsed by `_bool_expr`, which recognises `==` and `!=` after parsing the left-hand arithmetic expression. As a result, equality parsing is condition-focused in the current grammar implementation rather than a general infix operator available in every expression position. Function calls are recognised when an identifier is followed by `(`, with argument lists parsed as comma-separated expressions. Parenthesized expressions are supported to override precedence. Unary negation (`-x`, `-5`) is also handled at the `_factor` level: a leading `-` token causes the parser to recursively parse the operand and return `BinaryOp(0 - operand)`, so the existing type rules for subtraction cover negation without any additional inference code.

```
def _expr(self) -> ASTNode:  # +,-
    ...
def _term(self) -> ASTNode:  # *,/
    ...
def _factor(self) -> ASTNode:  # unary -, literals, identifiers/calls,
    grouped expr
    ...
```

Figure 4.1 shows the AST produced for the statement `z = x + y * 2;`. Because `_term` handles `*` before `_expr` handles `+`, the multiplication sub-tree is nested one level deeper, encoding the higher precedence of `*` over `+` without any explicit priority table at runtime.

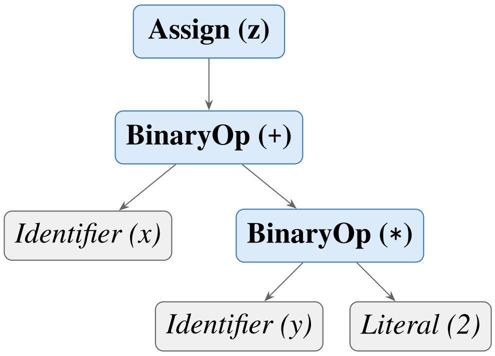


Figure 4.1. AST for `z = x + y * 2;`. Multiplication is a deeper sub-tree than addition, encoding higher precedence through tree structure rather than a runtime priority table.

Expression parsing is intentionally split into layered methods to enforce precedence and associativity in a transparent way. Parenthesized expressions are parsed explicitly and re-enter `_expr`, allowing precedence override where required by source syntax. Boolean conditions for `if` and `while` are parsed through `_bool_expr`, which accepts equality and inequality comparisons but can also return a non-comparison expression node; semantic enforcement that a condition is truly Boolean is deferred to the type checker.

4.2.3 Helper functions and lookahead

As described above, the parser uses `_check`, `_expect`, `_peek`, `_peek_next`, `_advance`, and `_is_at_end` for token navigation and validation. Lookahead is central for ambiguity resolution: assignment versus function call is resolved by inspecting the next token; comparison operators are recognised after parsing the left expression in `_bool_expr`. Parser errors follow a consistent diagnostic format: `[line X, col Y] ParseError: <message>`.

The parser also includes dedicated list parsers for formal parameters (`_params`) and call arguments (`_args`), both implemented as comma-separated sequences with optional emptiness. This ensures consistent handling of signatures such as `def f()` and `def f(a, b)` as well as invocations like `f()` and `f(x, y+1)`.

4.3 AST printing

For debugging and reporting, an AST printer converts the tree into a readable indented string, integrated into the pipeline output. The printer uses visitor-style dynamic dispatch, mapping node class names to methods such as `_visit_If` and `_visit_BinaryOp`, with output built as multi-line text where indentation reflects tree depth. Node formatting emphasises structure:

`Program` and `Block` include statement counts, `BinaryOp` explicitly shows left and right subtrees, and `FunctionCall` prints each argument by index. `Literal` includes both value and runtime Python type names for clarity.

4.4 Parser and type checker boundary

A deliberate architectural boundary is maintained between syntax construction and semantic validation. The parser is responsible for producing a structurally valid AST with accurate source locations, while type correctness and semantic constraints are checked later by the type checker. For example, `_bool_expr` may syntactically accept a non-Boolean expression as a condition node, but rejection of non-Boolean control-flow conditions is handled during semantic analysis. This separation keeps parser logic focused on grammar conformance and allows a clear responsibility split across team member contributions.

CHAPTER 5

TYPE INFERENCE ALGORITHM

5.1 Overview

Type inference in this project means that the programmer does not write explicit type declarations for variables or functions. Instead, the compiler infers types from assignments, literals, operations, function calls, and return statements. This keeps the language simple while still preserving static type safety.

The type checker runs after parsing. It receives the AST and validates the program before execution. Any inconsistent use of types is reported as a type error, and execution is stopped.

The interpreter stage does not repeat this validation: it walks the same AST with ordinary Python representations of values. Static type information remains in the symbol table produced by the checker; run-time values are not separately labelled with `LanguageType`, except that `print()` formats output by inspecting the value's Python type.

5.2 Type Environment

The type environment is represented by the symbol table. Conceptually, it is a mapping

$$\Gamma : \mathit{Identifier} \rightarrow \mathit{Type}$$

The environment grows as the checker visits the AST. On first assignment, a variable name is inserted with its inferred type. When a new block or a function call is entered, a child environment is created so that nested scopes can be checked independently while still accessing outer bindings when needed.

5.3 Type Inference Procedure

The type checker performs type inference by traversing the abstract syntax tree before interpretation. The algorithm is syntax-directed: each AST node is visited, the types of its child nodes are inferred first, and the current node is checked according to the language typing rules.

Algorithm $\textsc{TypeCheck}(\mathit{node}, \Gamma)$

Input: *node* = current AST node; Γ = current type environment / symbol table

Output: inferred type of *node*, or `TypeError`

1. **If** *node* is a **Literal**:   return the corresponding primitive type: `Integer`, `Float`, `Boolean`, or `String`.
2. **If** *node* is an **Identifier**:   look up the identifier in Γ. If it is not found, report an undefined variable error. Otherwise, return its stored type.
3. **If** *node* is a **BinaryOp**:
   - Infer the type of the left operand and the right operand.
   - If the operator is `+`, `-`, `*`, or `/`:   require both operands to be numeric; infer the result using Table 5.1.
   - If the operator is == or `!=`:   require both operands to be arithmetic; return `Boolean`.
   - Otherwise, report a `TypeError`.
4. **If** *node* is an **Assignment**:
   - Infer the type of the right-hand-side expression.
   - If the variable is not yet in Γ: insert it with the inferred type.
   - Else: verify the new type matches the existing type; if not, report a `TypeError`.
5. **If** *node* is an **If** statement:
   - Infer the condition type; require it to be `Boolean`.
   - Type-check the then-block in a child environment.
   - If an else-block exists, type-check it in a child environment.
6. **If** *node* is a **While** statement:
   - Infer the condition type; require it to be `Boolean`.
   - Type-check the loop body in a child environment.
7. **If** *node* is a **FunctionDef**:
   - Register the function name in Γ.
   - Parameter types are initially unknown.
   - Return type is inferred from `return` statements in the body.
8. **If** *node* is a **FunctionCall**:

- Infer all argument types.
- If this is the first valid call: store the argument types as the function's parameter types.
- Else: compare argument types with the stored parameter types.
- Type-check the function body using a local environment.
- Return the inferred function return type.

9. **If** *node* is a **Print** statement: infer the expression type; accept any of the four supported types.

5.4 Inference Rules

5.4.1 *Literals*

Literal nodes have direct types:

$$\begin{aligned} \Gamma \vdash 42 &: \texttt{Integer} \\ \Gamma \vdash 3.14 &: \texttt{Float} \\ \Gamma \vdash \texttt{true} &: \texttt{Boolean} \\ \Gamma \vdash \text{``}\texttt{hi}\text{''} &: \texttt{String} \end{aligned}$$

5.4.2 *No Operator Overloading*

The language does not support operator overloading. Every operator has a single, fixed meaning determined solely by the operator symbol. Specifically:

- The `+` operator is not overloaded for string concatenation. Applying `+` to a `String` operand is a type error.
- The comparison operators `==` and `!=` are not overloaded for string or boolean equality. Both operands must be arithmetic.
- No mixed Boolean or String arithmetic is permitted.

The apparent exception — that `Integer + Float` yields `Float` — is standard numeric promotion (widening coercion), not operator overloading. The + operator retains a single semantic meaning (arithmetic addition on numbers); only the result type widens to accommodate the wider operand. Overloading would mean assigning an unrelated meaning to the same symbol, such as using + for both numeric addition and string concatenation, which this language explicitly rejects. Because operators have a single, unambiguous type derivable from their operands, the compiler can always infer expression types without requiring explicit type declarations from the programmer.

5.4.3 Arithmetic Expressions

The arithmetic operators `+`, `-`, `*`, and `/` accept only numeric operands. If both operands are `Integer`, the result is `Integer`, except for division, which always produces `Float` regardless of the operand types. If at least one operand is `Float` for `+`, `-`, or `*`, the result is `Float` — this is numeric promotion, not operator overloading, because `+` retains a single numeric meaning throughout. Any non-numeric operand causes a type error.

Table 5.1. Arithmetic result types by operand combination

Left operand	Operator	Right operand	Result type
`Integer`	`+`, `-`, `*`	`Integer`	`Integer`
`Integer`	`/`	`Integer`	`Float`
`Float`	`+`, `-`, `*`, `/`	`Integer`	`Float`
`Integer`	`+`, `-`, `*`, `/`	`Float`	`Float`
`Float`	`+`, `-`, `*`, `/`	`Float`	`Float`
Any	any	`String` or `Boolean`	`TypeError`

Unary negation

Unary minus (`-x`) is desugared by the parser into `0 - x`. The type checker therefore applies the standard binary subtraction rule: if `x` is `Integer`, the result is `Integer`; if `x` is `Float`, the result is `Float`.

5.4.4 Boolean Expressions

The comparison operators `==` and `!=` always produce `Boolean`. Both operators require two arithmetic (`Integer` or `Float`) operands; this constraint is a deliberate design choice that keeps comparison semantics unambiguous. Comparisons involving `Boolean` or `String` operands are rejected by the type checker. Although `Boolean` and `String` values exist in the language, equality and inequality are deliberately restricted to arithmetic expressions because the project specification defines basic Boolean expressions as equality and inequality between two arithmetic expressions.

5.4.5 *Assignment Statement*

Assignments define variable types on first use. If a variable does not yet exist in the type environment, the checker infers the type of the right-hand side and inserts the variable into the environment. If the variable already exists, the new value must have the same type.

$$\text{If } x \notin \Gamma \text{ and } \Gamma \vdash e : T\text{, then } \Gamma, x : T \vdash x = e$$

$$\text{If } x : T \in \Gamma \text{ and } \Gamma \vdash e : T\text{, then } \Gamma \vdash x = e$$

5.4.6 *If-Then-Else*

The condition of an `if` statement must be `Boolean`. The then-block and else-block are checked in child scopes so that local operations can be validated cleanly.

$$\Gamma \vdash c : \texttt{Boolean} \quad \Gamma' \vdash B_{then} \quad \Gamma'' \vdash B_{else} \quad \Longrightarrow \quad \Gamma \vdash \texttt{if}\,(c)\,B_{then}\,\texttt{else}\,B_{else}$$

5.4.7 *While-Loop*

The condition of a `while` loop must also be `Boolean`. The loop body is checked in a child scope and must not violate the types of variables already introduced in the surrounding environment.

$$\Gamma \vdash c : \texttt{Boolean} \quad \Gamma' \vdash B \quad \Longrightarrow \quad \Gamma \vdash \texttt{while}\,(c)\,B$$

5.4.8 *Function Definition and Call*

Function parameter types are inferred from the first call to the function. After that first call, subsequent calls must use arguments of the same types. The return type is inferred from the `return` statements in the function body. If the function returns inconsistent types, the checker reports an error.

For functions that are defined but never called, the type checker performs a post-pass after all top-level statements have been processed. It infers parameter types by scanning the body for arithmetic and comparison usage, then checks the full body with those inferred types. This ensures that type errors inside never-called function bodies are still detected before execution.

Recursive calls are intentionally outside the scope of this implementation; supporting them would require additional fixed-point inference or explicit recursion handling that is orthogonal to the core type-inference algorithm.

5.5 Type Error Reporting

Type errors are reported with source positions so that the user can locate the problem quickly. The format used by the system is:

```
[line X, col Y] TypeError: message
```

Examples include assigning a `String` to an `Integer` variable, using a non-Boolean condition in an `if` statement, or supplying the wrong argument type to a function.

5.6 Example Type Inference Traces

Consider the following program fragment:

```
x = 10;
y = 20.5;
z = x + y;
print(z);
```

The checker infers:

- `x: Integer`
- `y: Float`
- `x + y: Float`
- `z: Float`

For a function example:

```
def add(a, b) {
        return a + b;
}

result = add(2, 3.5);
```

The first function call infers `a: Integer` and `b: Float`. The expression `a + b` is therefore `Float`, so the function return type is inferred as `Float`. The variable `result` is also inferred as `Float`.

CHAPTER 6

SYSTEM ARCHITECTURE AND IMPLEMENTATION

This chapter describes how the implementation is organised: the end-to-end processing flow, repository layout, technology choices, and the main components that realise static analysis and execution (type checker, interpreter, and user interfaces). Detailed treatment of lexical analysis, the symbol table, and parsing appears in Chapters 3 and 4; Chapter 5 presents the type-inference rules that the type checker implements.

6.1 Compiler pipeline

The system follows a four-stage pipeline. Lexical and syntactic processing are kept separate from semantic analysis and execution, which simplifies testing and localises errors to the appropriate stage.

The overall flow of data through the system is shown in Figure 6.1.

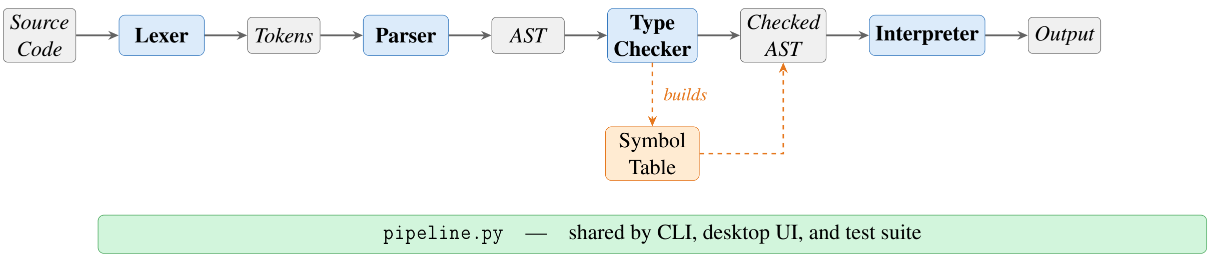


Figure 6.1. Compiler pipeline overview.

Source characters are scanned by the lexer into a token stream. The parser consumes tokens and builds an AST that reflects program structure (expressions, statements, and control flow). The type checker then walks the AST, populates and uses the symbol table, infers types, and validates scope and type rules. Only programs that pass the type checker proceed to interpretation. The interpreter evaluates the checked tree using ordinary Python values; it does not re-run the type rules, since correctness at runtime is guaranteed by the prior static pass. All four stages are orchestrated through `pipeline.py`, which is shared by the command-line runner, the desktop UI, and the test suite.

6.1.1 *End-to-end pipeline walkthrough*

The following example traces the complete execution of a three-statement program through all four pipeline stages, illustrating the correspondence between source text, tokens, abstract syntax tree, inferred types, and runtime output.

Source program:

```
x = 10;
y = 20.5;
print(x + y);
```

Stage 1 — Lexer output (token stream):

```
IDENTIFIER      'x'      line 1, col 1
ASSIGN          '='      line 1, col 3
INT_LITERAL     '10'     line 1, col 5
SEMICOLON       ';'      line 1, col 7
IDENTIFIER      'y'      line 2, col 1
ASSIGN          '='      line 2, col 3
FLOAT_LITERAL   '20.5'   line 2, col 5
SEMICOLON       ';'      line 2, col 9
PRINT           'print'  line 3, col 1
LPAREN          '('      line 3, col 6
IDENTIFIER      'x'      line 3, col 7
PLUS            '+'      line 3, col 9
IDENTIFIER      'y'      line 3, col 11
RPAREN          ')'      line 3, col 12
SEMICOLON       ';'      line 3, col 13
EOF                      line 3, col 14
```

Stage 2 — Parser output (abstract syntax tree):

```
Program  [3 statement(s)]
    Assign  'x'  =
        Literal  10  (int)
    Assign  'y'  =
        Literal  20.5  (float)
    Print
        BinaryOp  '+'
            left:
                Identifier  'x'
            right:
                Identifier  'y'
```

Stage 3 — Type checker output (inferred type environment):

Name	Inferred type	
`x`	`Integer`	
`y`	`Float`	
`x + y`	`Float`	(Integer $\times$ Float $\rightarrow$ Float)

Stage 4 — Interpreter output:

```
30.5 : Float
```

The example demonstrates the full data flow: source characters are tokenised by the lexer, structured into an AST by the parser, validated and annotated by the type checker, and finally evaluated by the interpreter to produce typed output.

6.1.2 Entry points and shared pipeline

All stages are orchestrated by `components/pipeline.py`, which is shared by the command-line runner, the desktop UI, and the test suite. From the project root, `python3 main.py` runs a script (or built-in sample) and prints the stages in order: tokens, AST text, type table, and execution output. `python3 ui.py` runs the Tkinter workbench: the same pipeline is used internally, with each stage shown in a separate tab so the user can inspect results without altering the underlying flow.

6.2 Project structure

The main source files and their responsibilities are summarised in Table 6.1.

Table 6.1. Main source files

File	Responsibility
`main.py`	Command-line runner for sample code or a script file
`ui.py`	Desktop UI for editing and running a script
`components/tokens.py`	Token classes and token categories
`components/lexica.py`	Lexical analysis
`components/ast_nodes.py`	AST node definitions
`components/parser.py`	Recursive-descent parser
`components/ast_printer.py`	AST renderer for debugging and reporting
`components/symbol_table.py`	Scoped symbol table and semantic types
`components/type_checker.py`	Static type checker and inference
`components/interpreter.py`	Tree-walking interpreter
`components/pipeline.py`	Shared pipeline for CLI, UI, and tests

6.3 Technology stack

The project uses Python and standard-library tools only (Table 6.2).

Table 6.2. Technology stack

Tool	Purpose
Python 3.10+	Core implementation language
`dataclasses`	AST and runtime helper structures
`tkinter`	Desktop graphical interface
`unittest`	Automated testing

6.4 Type checker

The `TypeChecker` class implements the inference and checking rules described in Chapter 5. It traverses the AST before execution, uses the symbol table to record variable and function information, infers types on first assignment, validates arithmetic and comparison expressions, enforces Boolean conditions for control flow, infers function parameter types from the first call (or from body usage for uncalled functions), and infers function return types from `return` statements. Programs that fail these rules are rejected before interpretation.

6.4.1 Scope of the implemented type system

In scope terms, this component turns a syntactically valid AST into a semantically valid program:

- static typing rules for arithmetic, comparison, assignment, control flow, and functions,
- type checking for expressions and statements,
- variable type inference from first assignment,
- function parameter inference from the first call (or from body usage for uncalled functions),
- function return type inference from `return` statements,
- source-positioned type errors when a program violates a typing rule.

6.5 Interpreter

The interpreter is a tree-walking evaluator over the checked AST. It maintains nested runtime environments for variables and function calls. Assignments write through the current or nearest enclosing scope; `if` selects a branch; `while` repeats until its condition becomes false; function calls bind arguments by value and return through an internal return mechanism.

The built-in `print()` function outputs both value and runtime type. All four types are supported:

```
print(42);         (* 42  : Integer       *)
print(3.14);       (* 3.14 : Float        *)
print(true);       (* true : Boolean      *)
print("plc");    (* "plc" : String      *)
```

6.5.1 *Scope of the implemented runtime*

The runtime stage provides observable behaviour and integrates with the shared pipeline:

- assignment execution,
- `if` execution,
- `while` execution,
- function execution with value parameter passing,
- `print()` execution with immediate output,
- execution behind the same `run_pipeline` path used by the CLI, UI, and tests.

6.6 Graphical user interface

The desktop workbench (`ui.py`) is implemented with Python's `tkinter` standard library and requires no additional dependencies. The window is titled *125970-126690-126687-Ass2-PLC* and opens at 1200×760 pixels in a horizontal split layout.

Toolbar. A top toolbar provides three buttons — *Open Script*, *Run*, and *Clear Output* — and a right-aligned status label that reports *Ready*, a loaded filename, or *Run failed*.

Left panel (source editor). A monospaced `Text` widget with both horizontal and vertical scrollbars allows the user to enter or paste arbitrary source code. The *Open Script* button populates the editor from a file. The editor is pre-loaded with the built-in sample program on first launch.

Right panel (output tabs). A `Notebook` widget presents four tabs, each backed by a scrollable read-only `Text` widget with both horizontal and vertical scrollbars:

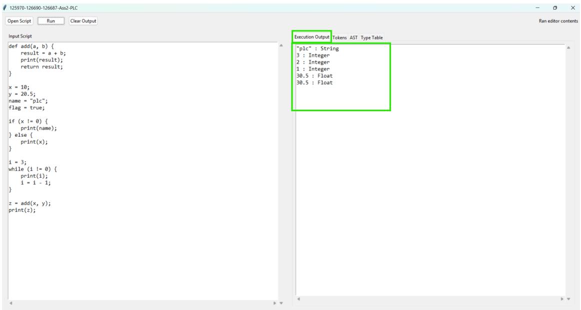
- *Execution Output* — the `print()` values produced by stage 4, formatted as `value : Type`;
- *Tokens* — the token stream from stage 1, one token per line with type, lexeme, and source position;
- *AST* — the indented tree text from stage 2;
- *Type Table* — the symbol table from stage 3 showing each name, kind, inferred type, initialisation state, and parameter list.

Error handling. If any pipeline stage raises an exception, the error message is written to the *Execution Output* tab as `ERROR: <message>`. No modal dialogs are used, so the user can edit and rerun without dismissing a popup.

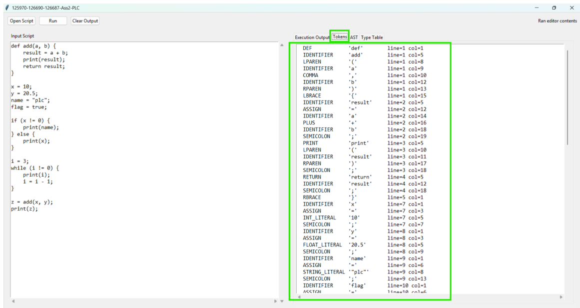
Keyboard shortcuts. `F5` and `Ctrl+Enter` are both bound to the *Run* action, allowing rapid edit-run cycles without reaching for the mouse.

6.6.1 Screenshots

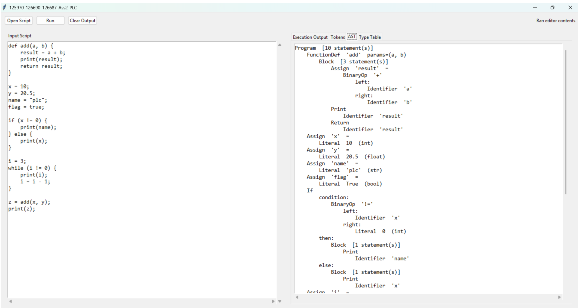
Figures 6.2 and 6.3 show the desktop workbench and CLI output.



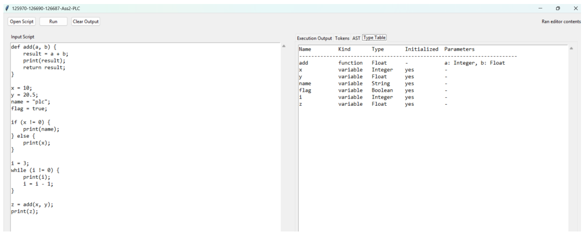
(a) Execution Output tab.



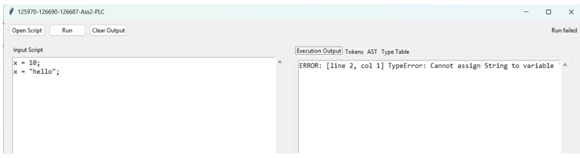
(b) Tokens tab (Stage 1).



(c) AST tab (Stage 2).



(d) Type Table tab (Stage 3).

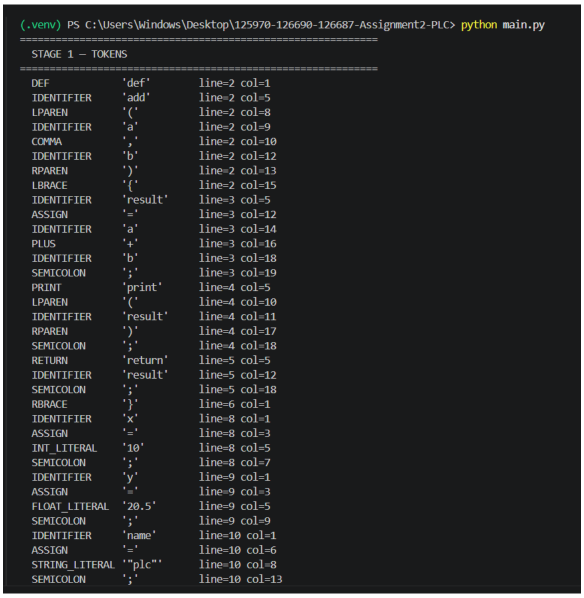


(e) Inline type-error display.

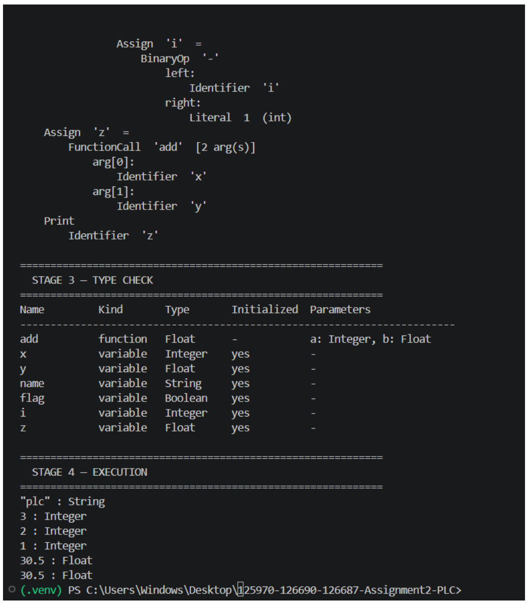
Figure 6.2. Desktop workbench — output tabs and error display.

6.7 Command-line interface

The CLI runner (`main.py`) accepts an optional script path; without one it runs the built-in sample and prints four labelled stages to `stdout`.



(a) Stage 1: token stream.



(b) Stage 3 type table and Stage 4 output.

Figure 6.3. CLI run of `python main.py` on the built-in sample.

CHAPTER 7

TESTING AND VERIFICATION

7.1 Manual Testing

Prior to the construction of the automated suite, each language feature was exercised manually by running the full pipeline and inspecting stage outputs. Table 7.1 summarises the categories covered.

Table 7.1. Representative manually tested categories

Category	Program Behaviour	Expected Result
Arithmetic	Evaluate mixed numeric expressions	Correct value and inferred type
If-then-else	Choose one branch from a Boolean condition	Only matching branch runs
While-loop	Print during repeated execution	One output line per iteration
Functions	Infer parameters and return type	Valid call and correct return value
Type mismatch	Reassign incompatible type	Type error before runtime

7.1.1 Lexer and Symbol Table

The lexer was verified manually by running the full pipeline and inspecting Stage 1 token output for programs that exercise every token category: integer and float literals, Boolean literals, string literals, all six keywords, two-character comparison operators (`==`, `!=`), single-character operators, and delimiters. Identifier scanning with keyword fallback was confirmed by checking that `true`, `false`, and reserved words are emitted with their keyword token types while non-keyword names produce `IDENTIFIER` tokens. For example, the source fragment `x = 10;` produces the token sequence:

```
IDENTIFIER  'x'   line=1 col=1
ASSIGN      '='   line=1 col=3
INT_LITERAL '10'  line=1 col=5
SEMICOLON   ';'   line=1 col=7
```

Error paths were also checked: supplying a bare `!` character raises a `LexerError` with a source position and a message suggesting `!=`, and an unterminated string literal raises a `LexerError` citing the opening line.

The symbol table was verified through the type checker and the Stage 3 output. Duplicate variable definitions were confirmed to raise a `SymbolTableError` before execution. Scope isolation was verified with nested blocks: a variable defined inside an `if` or `while` body is not visible in the outer scope after the block ends. The `format_table()` output was checked against the expected column layout (name, kind, type, initialisation status, parameters) for both variable and function symbols.

7.1.2 Parser and AST

The parser was verified manually by running the full pipeline and inspecting Stage 2 AST output. The following cases were checked:

- arithmetic expressions with mixed precedence (e.g. `x + y * 2`) produce a tree where multiplication is a deeper sub-tree than addition, confirming the `_expr`/`_term`/`_factor` precedence encoding,
- assignment versus expression statement dispatch: `x = 5;` produces an `Assign` node while `add(1,2);` produces a `FunctionCall` node at the statement level, confirming the IDENTIFIER+ASSIGN lookahead,
- `if` with and without an `else` clause: the resulting `If` node has a non-null `else_block` only when the clause is present,
- function definitions store only parameter names in the `FunctionDef` node; the AST printer output was inspected to confirm no type annotations are present at this stage,
- string literals in `Literal` nodes have their surrounding double-quotes stripped; `print("hello")` prints `"hello" : String` at runtime (the interpreter re-adds the quotes when formatting string output),
- parse errors: missing semicolons, unmatched parentheses, and unexpected tokens all produce `[line X, col Y] ParseError:` messages with correct source positions.

7.1.3 Arithmetic Expressions

Arithmetic expressions were tested with integer-only and mixed integer–float programs to verify operator precedence and numeric promotion. The expression `x + y * 2` confirmed that multiplication binds more tightly than addition, and mixed-type operands consistently produced `Float` results in accordance with the promotion rules defined in Chapter 5.

7.1.4 Boolean Expressions

Boolean expressions were tested inside `if` and `while` conditions. The type checker correctly enforced that both operands of `==` and `!=` must be arithmetic; supplying a `Boolean` or `String` operand produced a source-positioned `TypeCheckError` before any execution.

7.1.5 Assignment Statements

Assignments were tested for both first-use type inference and reassignment consistency. Assigning `5` to a variable and subsequently assigning `"hello"` to the same variable correctly produced a type error, confirming that the inferred type is fixed after first assignment.

7.1.6 If-Then-Else

Conditional programs were constructed to verify that exactly one branch executes for a given condition and that neither branch is entered when the condition evaluates to the opposite truth value. Non-Boolean conditions were also supplied and confirmed to be rejected by the type checker before execution.

7.1.7 While-Loop

While-loops were tested with an integer countdown from three to zero. The results confirmed that the loop body executes once per iteration and terminates when the condition becomes false, with `print()` producing one output line per pass rather than accumulating output until loop exit.

7.1.8 Functions

Function tests covered value parameter passing, local scope execution, and return value propagation. The type checker correctly inferred parameter types from the first call site and enforced that signature for all subsequent calls to the same function.

7.1.9 Print

The built-in `print()` function was tested with all supported runtime types. The output includes the printed value together with its type, such as `29.5 : Float` and `"plc" : String`.

7.1.10 Unary Minus

The unary minus operator was verified for integer literals (`x = -5`), float literals (`y = -3.14`), negated variable references (`y = -x`), and negation inside a larger expression (`x = 3 + -2`). In each case the type checker assigned the correct type and the interpreter produced the expected value.

7.2 Type Error Detection

The checker was verified with invalid programs involving incompatible assignment, invalid arithmetic operands, wrong function arguments, and non-Boolean conditions. In each case the system stopped before execution and reported a source-positioned type error. Representative error messages are shown below.

```
# Assignment type mismatch
[line 2, col 1] TypeError: Cannot assign String to variable 'x' of type
    Integer

# Non-Boolean if condition
[line 1, col 4] TypeError: If condition must have type Boolean

# Wrong argument type for function
[line 5, col 9] TypeError: Argument 1 of 'add' must be Integer, got
   String
```

7.3 Automated Test Suite

The automated regression suite is located in `tests/test_pipeline.py` and uses Python's built-in `unittest` framework. The tests are organised into five classes, each targeting a distinct compiler stage.

Table 7.2. Automated test coverage by class

Test class	Stage covered	Tests
`LexerTests`	Tokenisation (`lexica.py`)	10
`SymbolTableTests`	Scoped symbol table (`symbol_table.py`)	7
`ParserTests`	Parsing and precedence (`parser.py`)	5
`TypeCheckerTests`	Static type inference (`type_checker.py`)	15
`IntegrationTests`	Full pipeline execution	17
	Total	**54**

LexerTests verify that each token category (integer, float, Boolean, string, keywords, identifiers, two-character operators) is produced correctly, that source line numbers are tracked across newlines, and that illegal characters and unterminated strings raise `LexerError`.

SymbolTableTests exercise the symbol table directly, without going through the full pipeline. They cover variable definition and lookup, duplicate definition raising `SymbolTableError` for both variables and functions, parent-scope lookup through `child_scope()`, undefined-symbol lookup error, and the `format_table()` output for both variable and function symbol rows.

TypeCheckerTests verify static type inference and error detection. The 15 tests cover integer and float arithmetic, division producing `Float`, numeric promotion, string and boolean type inference, assignment type mismatch, non-Boolean conditions, undefined variables, wrong argument count and type, and two tests for uncalled-function body checking: one confirms that a type error inside a never-called function body is detected, and another confirms that a valid uncalled function raises no error.

IntegrationTests run programs through all four pipeline stages and verify the final output. Cases include unary negation on all numeric types, both branches of `if/else`, while-loop countdown, `print()` with every type, value-parameter isolation, nested function calls, variable reassignment, and scope isolation (variables declared inside an `if` block are not visible outside it).

The tests can be run with:

```
python3 -m unittest discover -s tests -v
```

All 54 tests pass.

7.4 Error Handling

The language system reports errors at three main stages:

- lexical errors for malformed characters or unterminated strings,
- parse errors for invalid syntax such as missing semicolons,
- type errors for programs that violate static typing rules.

Runtime errors are also reported with source positions whenever execution reaches an invalid state such as an undefined variable reference.

CHAPTER 8

CONCLUSION

This project delivered a complete, statically-typed interpreted language supporting Integer, Float, Boolean, and String types, with arithmetic, comparisons, assignments, if-then-else, while-loops, functions with value parameter passing, and a built-in `print()` function.

The system follows a four-stage pipeline: lexer, parser, type checker, and interpreter. Variable and function types are inferred without explicit declarations; programs that violate type rules are rejected before execution. For uncalled functions, the type checker performs a post-pass to infer parameter types from body usage, ensuring type errors are always caught.

The 54 automated tests across five classes — lexical analysis, symbol table, parsing, type checking, and full pipeline — confirm correct behaviour for all language features. Both a command-line runner and a desktop UI are provided.

The implementation demonstrates all core compiler-construction concepts covered in the course — lexical analysis, parsing, static type inference, and tree-walking interpretation — in a complete, tested, and working system. Future work could extend the language with recursive functions, richer data structures, and code generation.

REFERENCES

Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*. Addison-Wesley, Boston, MA, 2nd edition, 2006.

Robert Nystrom. *Crafting Interpreters*. Genever Benning, Seattle, WA, 2021.

Python Software Foundation. Python documentation: dataclasses — data classes. `https://docs.python.org/3/library/dataclasses.html`, 2026a. Accessed: April 2026.

Python Software Foundation. Python documentation: Tkinter — python interface to tcl/tk. `https://docs.python.org/3/library/tkinter.html`, 2026b. Accessed: April 2026.

Python Software Foundation. Python documentation: unittest — unit testing framework. `https://docs.python.org/3/library/unittest.html`, 2026c. Accessed: April 2026.

APPENDIX A: SOURCE CODE

The complete source code is available on GitHub:

https://github.com/The-Token-Trio/125970-126690-126687-Assignment2-PLC

The project can be run from the command line or the desktop UI:

```
python3 main.py
python3 ui.py
python3 -m unittest discover -s tests -v
```

No external Python dependencies are required.

APPENDIX B: TEAM MEMBERS AND CONTRIBUTIONS

Name	**Student ID**	**Contributions**
Aye Khin Khin Hpone (Yolanda Lim)	st125970	Static typing rules; type checking; assignment, if, while, function, and `print()` execution; unary minus inference; pipeline integration (`pipeline.py`); automated test suite (54 tests)
Applegate T. Tun Oo	st126690	Lexer implementation (scanning, tokenisation, line/column tracking, error reporting); token definitions; keywords, operators, identifiers, and literals; symbol table (variable/function/type storage)
Win Htut Naing	st126687	Arithmetic and boolean expressions; assignment, if-then-else, while-loop, function definition/call, and `print()` parsing; unary minus parsing

A.1 Entry Points

```
from __future__ import annotations

import argparse
from pathlib import Path

from components.pipeline import format_stage_output, run_pipeline


```

```
DEFAULT_SOURCE = """
def add(a, b) {
    result = a + b;
    print(result);
    return result;
}

x = 10;
y = 20.5;
name = "plc";
flag = true;

if (x != 0) {
    print(name);
} else {
    print(x);
}

i = 3;
while (i != 0) {
    print(i);
    i = i - 1;
}

z = add(x, y);
print(z);
"""


def run_source(source_code: str) -> None:
    result = run_pipeline(source_code)
    print(format_stage_output(result))


def _parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="Run the programming
        language pipeline.")
    parser.add_argument("script", nargs="?", help="Optional path to a
        source file to run")
    return parser.parse_args()


def main() -> None:
    args = _parse_args()
    if args.script:
        source_code = Path(args.script).read_text(encoding="utf-8")
    else:
        source_code = DEFAULT_SOURCE
    run_source(source_code)


if __name__ == "__main__":
    main()
```

Listing 8.1: main.py — command-line entry point

```
from __future__ import annotations

import tkinter as tk
from pathlib import Path
from tkinter import filedialog, ttk

from components.pipeline import format_tokens, run_pipeline


DEFAULT_SOURCE = """def add(a, b) {
    result = a + b;
    print(result);
    return result;
}

x = 10;
y = 20.5;
name = \"plc\";
flag = true;

if (x != 0) {
    print(name);
} else {
    print(x);
}

i = 3;
while (i != 0) {
    print(i);
    i = i - 1;
}

z = add(x, y);
print(z);
"""


class LanguageWorkbench:
    def __init__(self) -> None:
        self.root = tk.Tk()
        self.root.title("125970-126690-126687-Ass2-PLC")
        self.root.geometry("1200x760")
        self.current_file: Path | None = None

        self._build_layout()
        self.source_text.insert("1.0", DEFAULT_SOURCE)
        self.root.bind("<F5>", lambda _e: self._run_source())
        self.root.bind("<Control-Return>", lambda _e: self._run_source
            ())

```

```python
    def _build_layout(self) -> None:
        toolbar = ttk.Frame(self.root, padding=10)
        toolbar.pack(fill=tk.X)

        ttk.Button(toolbar, text="Open Script", command=self._open_script).pack(side=tk.LEFT)
        ttk.Button(toolbar, text="Run", command=self._run_source).pack(side=tk.LEFT, padx=(8, 0))
        ttk.Button(toolbar, text="Clear Output", command=self._clear_output).pack(side=tk.LEFT, padx=(8, 0))

        self.status_var = tk.StringVar(value="Ready")
        ttk.Label(toolbar, textvariable=self.status_var).pack(side=tk.RIGHT)

        body = ttk.Panedwindow(self.root, orient=tk.HORIZONTAL)
        body.pack(fill=tk.BOTH, expand=True, padx=10, pady=(0, 10))

        left_panel = ttk.Frame(body, padding=8)
        right_panel = ttk.Frame(body, padding=8)
        body.add(left_panel, weight=1)
        body.add(right_panel, weight=1)

        ttk.Label(left_panel, text="Input Script").pack(anchor=tk.W)
        src_frame = ttk.Frame(left_panel)
        src_frame.pack(fill=tk.BOTH, expand=True, pady=(6, 0))
        self.source_text = tk.Text(src_frame, wrap=tk.NONE, font=("Consolas", 11))
        src_vsb = ttk.Scrollbar(src_frame, orient=tk.VERTICAL, command=self.source_text.yview)
        src_hsb = ttk.Scrollbar(src_frame, orient=tk.HORIZONTAL, command=self.source_text.xview)
        self.source_text.configure(yscrollcommand=src_vsb.set, xscrollcommand=src_hsb.set)
        src_vsb.pack(side=tk.RIGHT, fill=tk.Y)
        src_hsb.pack(side=tk.BOTTOM, fill=tk.X)
        self.source_text.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)

        notebook = ttk.Notebook(right_panel)
        notebook.pack(fill=tk.BOTH, expand=True)

        self.output_text = self._make_output_tab(notebook, "Execution Output")
        self.tokens_text = self._make_output_tab(notebook, "Tokens")
        self.ast_text = self._make_output_tab(notebook, "AST")
        self.types_text = self._make_output_tab(notebook, "Type Table")

    def _make_output_tab(self, notebook: ttk.Notebook, title: str) -> tk.Text:
        frame = ttk.Frame(notebook, padding=8)
        notebook.add(frame, text=title)
        text = tk.Text(frame, wrap=tk.NONE, font=("Consolas", 11), state=tk.DISABLED)
```

```python
        vsb = ttk.Scrollbar(frame, orient=tk.VERTICAL, command=text.yview)
        hsb = ttk.Scrollbar(frame, orient=tk.HORIZONTAL, command=text.xview)
        text.configure(yscrollcommand=vsb.set, xscrollcommand=hsb.set)
        vsb.pack(side=tk.RIGHT, fill=tk.Y)
        hsb.pack(side=tk.BOTTOM, fill=tk.X)
        text.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)
        return text

    def _open_script(self) -> None:
        file_path = filedialog.askopenfilename(
            title="Open Script File",
            filetypes=[("Text Files", "*.txt *.pl *.src"), ("All Files", "*.*")],
        )
        if not file_path:
            return

        path = Path(file_path)
        self.current_file = path
        self.source_text.delete("1.0", tk.END)
        self.source_text.insert("1.0", path.read_text(encoding="utf-8"))
        self.status_var.set(f"Loaded {path.name}")

    def _run_source(self) -> None:
        source = self.source_text.get("1.0", tk.END)
        try:
            result = run_pipeline(source)
        except Exception as error:
            self._write_text(self.output_text, f"ERROR: {error}")
            self.status_var.set("Run failed")
            return

        self._write_text(self.output_text, "\n".join(result.outputs) if result.outputs else "(no output)")
        self._write_text(self.tokens_text, format_tokens(result.tokens))
        self._write_text(self.ast_text, result.ast_text)
        self._write_text(self.types_text, result.checked_scope.format_table())

        current_label = self.current_file.name if self.current_file is not None else "editor contents"
        self.status_var.set(f"Ran {current_label}")

    def _clear_output(self) -> None:
        self._write_text(self.output_text, "")
        self._write_text(self.tokens_text, "")
        self._write_text(self.ast_text, "")
        self._write_text(self.types_text, "")
        self.status_var.set("Output cleared")
```

```python

    @staticmethod
    def _write_text(widget: tk.Text, content: str) -> None:
        widget.config(state=tk.NORMAL)
        widget.delete("1.0", tk.END)
        widget.insert("1.0", content)
        widget.config(state=tk.DISABLED)

    def run(self) -> None:
        self.root.mainloop()


if __name__ == "__main__":
    LanguageWorkbench().run()
```

Listing 8.2: ui.py — Tkinter desktop workbench

A.2 Core Components

```python
from __future__ import annotations

from dataclasses import dataclass
from enum import Enum, auto


class TokenType(Enum):
    # Literals
    INT_LITERAL = auto()
    FLOAT_LITERAL = auto()
    BOOL_LITERAL = auto()
    STRING_LITERAL = auto()

    # Identifier
    IDENTIFIER = auto()

    # Keywords
    IF = auto()
    ELSE = auto()
    WHILE = auto()
    DEF = auto()
    RETURN = auto()
    PRINT = auto()

    # Operators
    PLUS = auto()
    MINUS = auto()
    STAR = auto()
    SLASH = auto()
    EQUAL_EQUAL = auto()
    BANG_EQUAL = auto()
    ASSIGN = auto()

```

```python
    # Delimiters
    LPAREN = auto()
    RPAREN = auto()
    LBRACE = auto()
    RBRACE = auto()
    COMMA = auto()
    SEMICOLON = auto()

    EOF = auto()


@dataclass(frozen=True)
class Token:
    token_type: TokenType
    lexeme: str
    line: int
    column: int
```

Listing 8.3: components/tokens.py — token type enumeration and Token dataclass

```python
from __future__ import annotations

from typing import Iterator

from components.tokens import Token, TokenType


KEYWORDS: dict[str, TokenType] = {
    "if": TokenType.IF,
    "else": TokenType.ELSE,
    "while": TokenType.WHILE,
    "def": TokenType.DEF,
    "return": TokenType.RETURN,
    "print": TokenType.PRINT,
    "true": TokenType.BOOL_LITERAL,
    "false": TokenType.BOOL_LITERAL,
}


SINGLE_CHAR_TOKENS: dict[str, TokenType] = {
    "+": TokenType.PLUS,
    "-": TokenType.MINUS,
    "*": TokenType.STAR,
    "/": TokenType.SLASH,
    "=": TokenType.ASSIGN,
    "(": TokenType.LPAREN,
    ")": TokenType.RPAREN,
    "{": TokenType.LBRACE,
    "}": TokenType.RBRACE,
    ",": TokenType.COMMA,
    ";": TokenType.SEMICOLON,
}

```

```python

class LexerError(Exception):
    pass


class Lexer:
    def __init__(self, source: str) -> None:
        self.source = source
        self.start = 0
        self.current = 0
        self.line = 1
        self.column = 1
        self.token_column = 1

    def tokenize(self) -> list[Token]:
        tokens: list[Token] = []
        while not self._is_at_end():
            self.start = self.current
            self.token_column = self.column
            token = self._scan_token()
            if token is not None:
                tokens.append(token)
        tokens.append(Token(TokenType.EOF, "", self.line, self.column))
        return tokens

    def _scan_token(self) -> Token | None:
        char = self._advance()

        # Whitespace handling
        if char in {" ", "\t", "\r"}:
            return None
        if char == "\n":
            self.line += 1
            self.column = 1
            return None

        # Two-char operators
        if char == "=":
            if self._match("="):
                return self._make_token(TokenType.EQUAL_EQUAL)
            return self._make_token(TokenType.ASSIGN)
        if char == "!":
            if self._match("="):
                return self._make_token(TokenType.BANG_EQUAL)
            raise LexerError(self._error_message("Unexpected '!'. Did
                you mean '!='?"))

        # Single-char operators/delimiters
        single_char_token = SINGLE_CHAR_TOKENS.get(char)
        if single_char_token is not None:
            return self._make_token(single_char_token)

        if char == '"':
```

```
            return self._string()

        if char.isdigit():
            return self._number()

        if self._is_identifier_start(char):
            return self._identifier()

        raise LexerError(self._error_message(f"Unexpected character: {char!r}"))

    def _string(self) -> Token:
        while self._peek() != '"' and not self._is_at_end():
            if self._peek() == "\n":
                self.line += 1
                self.column = 1
            self._advance()

        if self._is_at_end():
            raise LexerError(self._error_message("Unterminated string literal"))

        # Closing quote
        self._advance()
        return self._make_token(TokenType.STRING_LITERAL)

    def _number(self) -> Token:
        while self._peek().isdigit():
            self._advance()

        is_float = False
        if self._peek() == "." and self._peek_next().isdigit():
            is_float = True
            self._advance()
            while self._peek().isdigit():
                self._advance()

        if is_float:
            return self._make_token(TokenType.FLOAT_LITERAL)
        return self._make_token(TokenType.INT_LITERAL)

    def _identifier(self) -> Token:
        while self._is_identifier_part(self._peek()):
            self._advance()

        lexeme = self._current_lexeme()
        token_type = KEYWORDS.get(lexeme, TokenType.IDENTIFIER)
        return self._make_token(token_type)

    def _current_lexeme(self) -> str:
        return self.source[self.start : self.current]

    def _make_token(self, token_type: TokenType) -> Token:
```

```python
        return Token(token_type, self._current_lexeme(), self.line, self.token_column)

    def _advance(self) -> str:
        char = self.source[self.current]
        self.current += 1
        self.column += 1
        return char

    def _match(self, expected: str) -> bool:
        if self._is_at_end():
            return False
        if self.source[self.current] != expected:
            return False

        self.current += 1
        self.column += 1
        return True

    def _peek(self) -> str:
        if self._is_at_end():
            return "\0"
        return self.source[self.current]

    def _peek_next(self) -> str:
        if self.current + 1 >= len(self.source):
            return "\0"
        return self.source[self.current + 1]

    def _is_at_end(self) -> bool:
        return self.current >= len(self.source)

    @staticmethod
    def _is_identifier_start(char: str) -> bool:
        return char.isalpha() or char == "_"

    @staticmethod
    def _is_identifier_part(char: str) -> bool:
        return char.isalnum() or char == "_"

    def _error_message(self, message: str) -> str:
        return f"[line {self.line}, col {self.token_column}] {message}"


def iter_tokens(source: str) -> Iterator[Token]:
    yield from Lexer(source).tokenize()
```

Listing 8.4: components/lexica.py — lexical analyser

```python
from __future__ import annotations

from dataclasses import dataclass, field
from typing import Optional
```

```


# ---------------------------------------------------------------------------
# Base
# ---------------------------------------------------------------------------

@dataclass
class ASTNode:
    """Base class for every AST node. Carries source location for
       diagnostics."""
    line: int = field(default=0, kw_only=True)
    column: int = field(default=0, kw_only=True)


# ---------------------------------------------------------------------------
# Expressions
# ---------------------------------------------------------------------------

@dataclass
class Literal(ASTNode):
    """An integer, float, boolean, or string literal.

    Examples:  42   3.14   true   "hello"
    """
    value: int | float | bool | str


@dataclass
class Identifier(ASTNode):
    """A variable or function name used as an expression.

    Example:  x
    """
    name: str


@dataclass
class BinaryOp(ASTNode):
    """An arithmetic or comparison binary operation.

    op is one of: '+' '-' '*' '/' '==' '!='
    Example:  a + b   x == 0
    """
    op: str
    left: ASTNode
```

```python
    right: ASTNode


@dataclass
class FunctionCall(ASTNode):
    """A call to a user-defined function.

    Example:  add(1, 2)
    """
    name: str
    args: list[ASTNode] = field(default_factory=list)


# ---------------------------------------------------------------------------
# Statements
# ---------------------------------------------------------------------------

@dataclass
class Assign(ASTNode):
    """Variable assignment (first use infers type; later uses must
       match).

    Example:  x = 10;
    """
    name: str
    value: ASTNode


@dataclass
class Print(ASTNode):
    """Built-in print statement.

    Example:  print(x);
    """
    expr: ASTNode


@dataclass
class Return(ASTNode):
    """Return statement inside a function body.

    Example:  return result;
    """
    expr: ASTNode


@dataclass
class Block(ASTNode):
    """A brace-enclosed sequence of statements.
```

```

    Example:  { x = 1; print(x); }
    """
    statements: list[ASTNode] = field(default_factory=list)


@dataclass
class If(ASTNode):
    """If / optional else control flow.

    Example:  if (x != 0) { print(x); } else { print(0); }
    """
    condition: ASTNode
    then_block: Block
    else_block: Optional[Block]


@dataclass
class While(ASTNode):
    """While loop.

    Example:  while (x != 0) { x = x - 1; }
    """
    condition: ASTNode
    body: Block


@dataclass
class FunctionDef(ASTNode):
    """Function definition.

    Example:  def add(a, b) { return a + b; }

    Note: parameter types are unknown at parse time -- Member 3's type checker
    will infer them from usage. We store only the parameter names here.
    """
    name: str
    params: list[str]
    body: Block


# ---------------------------------------------------------------------------
# Top-level
# ---------------------------------------------------------------------------

@dataclass
class Program(ASTNode):
    """Root node: the entire source file as a list of statements."""
```

```
    statements: list[ASTNode] = field(default_factory=list)
```

Listing 8.5: components/ast_nodes.py — AST node definitions

```
from __future__ import annotations

from typing import Optional

from components.tokens import Token, TokenType
from components.ast_nodes import (
    ASTNode,
    Program,
    Block,
    Assign,
    If,
    While,
    FunctionDef,
    FunctionCall,
    Print,
    Return,
    BinaryOp,
    Literal,
    Identifier,
)


# ---------------------------------------------------------------------------
# Parser error
# ---------------------------------------------------------------------------

class ParseError(Exception):
    pass


# ---------------------------------------------------------------------------
# Parser
# ---------------------------------------------------------------------------

class Parser:
    """Recursive-descent parser.

    Usage:
        tokens = Lexer(source).tokenize()
        tree   = Parser(tokens).parse()   # returns Program node
    """
```

```python

    def __init__(self, tokens: list[Token]) -> None:
        self._tokens = tokens
        self._pos = 0

    # ---------------------------------------------------------------------------
    # Public entry point
    # ---------------------------------------------------------------------------

    def parse(self) -> Program:
        """Parse the full token stream and return the root Program node."""
        line, col = self._peek().line, self._peek().column
        statements: list[ASTNode] = []
        while not self._is_at_end():
            statements.append(self._statement())
        return Program(statements=statements, line=line, column=col)

    # ---------------------------------------------------------------------------
    # Statements
    # ---------------------------------------------------------------------------

    def _statement(self) -> ASTNode:
        """Dispatch to the correct statement parser based on the current token."""
        tok = self._peek()

        if tok.token_type == TokenType.DEF:
            return self._func_def()

        if tok.token_type == TokenType.IF:
            return self._if_stmt()

        if tok.token_type == TokenType.WHILE:
            return self._while_stmt()

        if tok.token_type == TokenType.RETURN:
            return self._return_stmt()

        if tok.token_type == TokenType.PRINT:
            return self._print_stmt()

        # assignment: IDENTIFIER "=" expr ";"
        if (
            tok.token_type == TokenType.IDENTIFIER
```

```
            and self._peek_next().token_type == TokenType.ASSIGN
        ):
            return self._assignment()

        # fallback: bare expression statement  expr ";"
        node = self._expr()
        self._expect(TokenType.SEMICOLON, "Expected ';' after expression")
        return node

    def _assignment(self) -> Assign:
        """IDENTIFIER '=' expr ';'"""
        name_tok = self._advance()                                       # IDENTIFIER
        self._advance()                                                  # '='
        value = self._expr()
        self._expect(TokenType.SEMICOLON, "Expected ';' after assignment")
        return Assign(name=name_tok.lexeme, value=value,
                      line=name_tok.line, column=name_tok.column)

    def _if_stmt(self) -> If:
        """'if' '(' bool_expr ')' block ( 'else' block )?"""
        tok = self._advance()                                            # 'if'
        self._expect(TokenType.LPAREN, "Expected '(' after 'if'")
        condition = self._bool_expr()
        self._expect(TokenType.RPAREN, "Expected ')' after if condition")
        then_block = self._block()
        else_block: Optional[Block] = None
        if self._check(TokenType.ELSE):
            self._advance()                                              # 'else'
            else_block = self._block()
        return If(condition=condition, then_block=then_block, else_block=else_block,
                  line=tok.line, column=tok.column)

    def _while_stmt(self) -> While:
        """'while' '(' bool_expr ')' block"""
        tok = self._advance()                                            # 'while'
        self._expect(TokenType.LPAREN, "Expected '(' after 'while'")
        condition = self._bool_expr()
        self._expect(TokenType.RPAREN, "Expected ')' after while condition")
        body = self._block()
        return While(condition=condition, body=body,
                     line=tok.line, column=tok.column)

    def _func_def(self) -> FunctionDef:
        """'def' IDENTIFIER '(' params? ')' block"""
        tok = self._advance()                                            # 'def'
```

```python
        name_tok = self._expect(TokenType.IDENTIFIER, "Expected function name after 'def'")
        self._expect(TokenType.LPAREN, "Expected '(' after function name")
        params = self._params()
        self._expect(TokenType.RPAREN, "Expected ')' after parameters")
        body = self._block()
        return FunctionDef(name=name_tok.lexeme, params=params, body=body,
                           line=tok.line, column=tok.column)

    def _print_stmt(self) -> Print:
        """'print' '(' expr ')' ';'"""
        tok = self._advance()                                   # 'print'
        self._expect(TokenType.LPAREN, "Expected '(' after 'print'")
        expr = self._expr()
        self._expect(TokenType.RPAREN, "Expected ')' after print expression")
        self._expect(TokenType.SEMICOLON, "Expected ';' after print statement")
        return Print(expr=expr, line=tok.line, column=tok.column)

    def _return_stmt(self) -> Return:
        """'return' expr ';'"""
        tok = self._advance()                                   # 'return'
        expr = self._expr()
        self._expect(TokenType.SEMICOLON, "Expected ';' after return value")
        return Return(expr=expr, line=tok.line, column=tok.column)

    def _block(self) -> Block:
        """'{' statement* '}'"""
        tok = self._expect(TokenType.LBRACE, "Expected '{'")
        statements: list[ASTNode] = []
        while not self._check(TokenType.RBRACE) and not self._is_at_end():
            statements.append(self._statement())
        self._expect(TokenType.RBRACE, "Expected '}' to close block")
        return Block(statements=statements, line=tok.line, column=tok.column)

    # -----------------------------------------------------------------------
    # Parameters (function definition)
    # -----------------------------------------------------------------------

    def _params(self) -> list[str]:
        """IDENTIFIER (',' IDENTIFIER)*  -- returns list of param names.
```

```
        """
        params: list[str] = []
        if self._check(TokenType.IDENTIFIER):
            params.append(self._advance().lexeme)
            while self._check(TokenType.COMMA):
                self._advance()                                  # ','
                tok = self._expect(TokenType.IDENTIFIER, "Expected parameter name after ','")
                params.append(tok.lexeme)
        return params

    # ----------------------------------------------------------------------
    # Boolean expressions  (lowest precedence, used in if/while conditions)
    # ----------------------------------------------------------------------

    def _bool_expr(self) -> ASTNode:
        """expr ('==' | '!=') expr   |   BOOL_LITERAL"""
        # bare boolean literal: true / false
        if self._check(TokenType.BOOL_LITERAL):
            tok = self._advance()
            value = tok.lexeme == "true"
            return Literal(value=value, line=tok.line, column=tok.column)

        left = self._expr()

        if self._check(TokenType.EQUAL_EQUAL):
            op_tok = self._advance()
            right = self._expr()
            return BinaryOp(op="==", left=left, right=right,
                            line=op_tok.line, column=op_tok.column)

        if self._check(TokenType.BANG_EQUAL):
            op_tok = self._advance()
            right = self._expr()
            return BinaryOp(op="!=", left=left, right=right,
                            line=op_tok.line, column=op_tok.column)

        # no comparison operator found -- return the expr as-is
        # (the type checker will validate it's Boolean)
        return left

    # ----------------------------------------------------------------------
    # Arithmetic expressions  (standard precedence)
    # ----------------------------------------------------------------------
```

```
    def _expr(self) -> ASTNode:
        """term (('+' | '-') term)*"""
        node = self._term()
        while self._check(TokenType.PLUS) or self._check(TokenType.MINUS):
            op_tok = self._advance()
            right = self._term()
            node = BinaryOp(op=op_tok.lexeme, left=node, right=right,
                            line=op_tok.line, column=op_tok.column)
        return node

    def _term(self) -> ASTNode:
        """factor (('*' | '/') factor)*"""
        node = self._factor()
        while self._check(TokenType.STAR) or self._check(TokenType.SLASH):
            op_tok = self._advance()
            right = self._factor()
            node = BinaryOp(op=op_tok.lexeme, left=node, right=right,
                            line=op_tok.line, column=op_tok.column)
        return node

    def _factor(self) -> ASTNode:
        """
        '-' factor  (unary negation, desugared to 0 - factor)
        | INT_LITERAL | FLOAT_LITERAL | STRING_LITERAL | BOOL_LITERAL
        | IDENTIFIER ( '(' args? ')' )?
        | '(' expr ')'
        """
        tok = self._peek()

        # Unary minus  '-' factor
        if tok.token_type == TokenType.MINUS:
            op_tok = self._advance()
            operand = self._factor()
            zero = Literal(value=0, line=op_tok.line, column=op_tok.column)
            return BinaryOp(op="-", left=zero, right=operand,
                            line=op_tok.line, column=op_tok.column)

        # Integer literal
        if tok.token_type == TokenType.INT_LITERAL:
            self._advance()
            return Literal(value=int(tok.lexeme), line=tok.line, column=tok.column)

        # Float literal
        if tok.token_type == TokenType.FLOAT_LITERAL:
            self._advance()
            return Literal(value=float(tok.lexeme), line=tok.line, column=tok.column)
```

```python

        # Boolean literal
        if tok.token_type == TokenType.BOOL_LITERAL:
            self._advance()
            return Literal(value=(tok.lexeme == "true"), line=tok.line, column=tok.column)

        # String literal  (keep the surrounding quotes stripped)
        if tok.token_type == TokenType.STRING_LITERAL:
            self._advance()
            return Literal(value=tok.lexeme[1:-1], line=tok.line, column=tok.column)

        # Identifier -- could be a plain variable or a function call
        if tok.token_type == TokenType.IDENTIFIER:
            self._advance()
            if self._check(TokenType.LPAREN):
                # function call
                self._advance()                                    # '('
                args = self._args()
                self._expect(TokenType.RPAREN, "Expected ')' after function arguments")
                return FunctionCall(name=tok.lexeme, args=args,
                                    line=tok.line, column=tok.column)
            return Identifier(name=tok.lexeme, line=tok.line, column=tok.column)

        # Grouped expression  '(' expr ')'
        if tok.token_type == TokenType.LPAREN:
            self._advance()                                        # '('
            node = self._expr()
            self._expect(TokenType.RPAREN, "Expected ')' to close grouped expression")
            return node

        raise ParseError(self._error_at(tok, f"Unexpected token: {tok.lexeme!r}"))

    # ----------------------------------------------------------------------
    # Arguments (function call)
    # ----------------------------------------------------------------------

    def _args(self) -> list[ASTNode]:
        """expr (',' expr)*"""
        args: list[ASTNode] = []
        if not self._check(TokenType.RPAREN):
            args.append(self._expr())
            while self._check(TokenType.COMMA):
                self._advance()                                    # ','
```

```python
                args.append(self._expr())
        return args

    # ----------------------------------------------------------------------

    # Token navigation helpers
    # ----------------------------------------------------------------------


    def _peek(self) -> Token:
        return self._tokens[self._pos]

    def _peek_next(self) -> Token:
        if self._pos + 1 < len(self._tokens):
            return self._tokens[self._pos + 1]
        return self._tokens[-1]                              # EOF

    def _advance(self) -> Token:
        tok = self._tokens[self._pos]
        if not self._is_at_end():
            self._pos += 1
        return tok

    def _check(self, token_type: TokenType) -> bool:
        return self._peek().token_type == token_type

    def _is_at_end(self) -> bool:
        return self._peek().token_type == TokenType.EOF

    def _expect(self, token_type: TokenType, message: str) -> Token:
        if self._check(token_type):
            return self._advance()
        raise ParseError(self._error_at(self._peek(), message))

    def _error_at(self, tok: Token, message: str) -> str:
        return f"[line {tok.line}, col {tok.column}] ParseError: {
            message}"
```

Listing 8.6: components/parser.py — recursive-descent parser

```python
from __future__ import annotations

from dataclasses import dataclass, field
from enum import Enum


class LanguageType(Enum):
    INTEGER = "Integer"
    FLOAT = "Float"
    BOOLEAN = "Boolean"
    STRING = "String"
```

```python
    VOID = "Void"


@dataclass
class Symbol:
    name: str
    symbol_type: LanguageType


@dataclass
class VariableSymbol(Symbol):
    initialized: bool = False


@dataclass
class FunctionSymbol(Symbol):
    parameters: list[tuple[str, LanguageType]] = field(default_factory=
        list)


class SymbolTableError(Exception):
    pass


class SymbolTable:
    def __init__(self, parent: SymbolTable | None = None) -> None:
        self.parent = parent
        self._symbols: dict[str, Symbol] = {}

    def define_variable(self, name: str, symbol_type: LanguageType,
        initialized: bool = False) -> VariableSymbol:
        if name in self._symbols:
            raise SymbolTableError(f"Symbol already defined in this
                scope: {name}")
        symbol = VariableSymbol(name=name, symbol_type=symbol_type,
            initialized=initialized)
        self._symbols[name] = symbol
        return symbol

    def define_function(
        self,
        name: str,
        return_type: LanguageType,
        parameters: list[tuple[str, LanguageType]],
    ) -> FunctionSymbol:
        if name in self._symbols:
            raise SymbolTableError(f"Symbol already defined in this
                scope: {name}")
        symbol = FunctionSymbol(name=name, symbol_type=return_type,
            parameters=parameters)
        self._symbols[name] = symbol
        return symbol

```

```python
    def lookup(self, name: str) -> Symbol:
        symbol = self._symbols.get(name)
        if symbol is not None:
            return symbol
        if self.parent is not None:
            return self.parent.lookup(name)
        raise SymbolTableError(f"Undefined symbol: {name}")

    def exists_in_current_scope(self, name: str) -> bool:
        return name in self._symbols

    def child_scope(self) -> SymbolTable:
        return SymbolTable(parent=self)

    def set_initialized(self, name: str) -> None:
        symbol = self.lookup(name)
        if not isinstance(symbol, VariableSymbol):
            raise SymbolTableError(f"Cannot initialize non-variable symbol: {name}")
        symbol.initialized = True

    def snapshot(self) -> dict[str, Symbol]:
        return dict(self._symbols)

    def format_table(self) -> str:
        lines = [
            f"{'Name':<12} {'Kind':<10} {'Type':<10} {'Initialized':<12} Parameters",
            "-" * 72,
        ]

        for name, symbol in self._symbols.items():
            if isinstance(symbol, FunctionSymbol):
                params = ", ".join(f"{param_name}: {param_type.value}" for param_name, param_type in symbol.parameters)
                lines.append(
                    f"{name:<12} {'function':<10} {symbol.symbol_type.value:<10} {'-':<12} {params or '-'}"
                )
            elif isinstance(symbol, VariableSymbol):
                initialized = "yes" if symbol.initialized else "no"
                lines.append(
                    f"{name:<12} {'variable':<10} {symbol.symbol_type.value:<10} {initialized:<12} -"
                )
            else:
                lines.append(f"{name:<12} {'symbol':<10} {symbol.symbol_type.value:<10} {'-':<12} -")

        return "\n".join(lines)
```

Listing 8.7: components/symbol_table.py — scoped symbol table

```
from __future__ import annotations

from dataclasses import dataclass

from components.ast_nodes import (
    ASTNode,
    Assign,
    BinaryOp,
    Block,
    FunctionCall,
    FunctionDef,
    Identifier,
    If,
    Literal,
    Print,
    Program,
    Return,
    While,
)
from components.symbol_table import FunctionSymbol, LanguageType,
   SymbolTable, SymbolTableError, VariableSymbol


class TypeCheckError(Exception):
    pass


@dataclass
class _FunctionState:
    node: FunctionDef
    body_checked: bool = False


@dataclass
class _FunctionContext:
    name: str
    return_type: LanguageType | None = None


class TypeChecker:
    def __init__(self) -> None:
        self.global_scope = SymbolTable()
        self._functions: dict[str, _FunctionState] = {}
        self._active_calls: list[str] = []

    def check(self, program: Program) -> SymbolTable:
        for statement in program.statements:
            if isinstance(statement, FunctionDef):
                self._register_function(statement)

        for statement in program.statements:
            if isinstance(statement, FunctionDef):
```

```python
                continue
            self._check_statement(statement, self.global_scope, None)

        self._check_uncalled_functions()
        return self.global_scope

    def _register_function(self, node: FunctionDef) -> None:
        if node.name in self._functions:
            raise TypeCheckError(self._error_at(node, f"Function '{node.name}' is already defined"))

        placeholder_parameters = [(param_name, LanguageType.VOID) for param_name in node.params]
        try:
            self.global_scope.define_function(node.name, LanguageType.VOID, placeholder_parameters)
        except SymbolTableError as error:
            raise TypeCheckError(self._error_at(node, str(error))) from error
        self._functions[node.name] = _FunctionState(node=node)

    def _check_statement(
        self,
        node: ASTNode,
        scope: SymbolTable,
        function_context: _FunctionContext | None,
    ) -> None:
        if isinstance(node, Assign):
            value_type = self._infer_expr_type(node.value, scope)
            self._assign_variable_type(scope, node, value_type)
            return

        if isinstance(node, Print):
            self._infer_expr_type(node.expr, scope)
            return

        if isinstance(node, Return):
            if function_context is None:
                raise TypeCheckError(self._error_at(node, "'return' is only valid inside a function"))
            expr_type = self._infer_expr_type(node.expr, scope)
            if function_context.return_type is None:
                function_context.return_type = expr_type
            elif function_context.return_type != expr_type:
                raise TypeCheckError(
                    self._error_at(
                        node,
                        f"Function '{function_context.name}' returns both {function_context.return_type.value} and {expr_type.value}",
                    )
                )
            return
```

```

        if isinstance(node, If):
            condition_type = self._infer_expr_type(node.condition, scope)
            if condition_type != LanguageType.BOOLEAN:
                raise TypeCheckError(self._error_at(node.condition, "If condition must have type Boolean"))
            self._check_block(node.then_block, scope.child_scope(), function_context)
            if node.else_block is not None:
                self._check_block(node.else_block, scope.child_scope(), function_context)
            return

        if isinstance(node, While):
            condition_type = self._infer_expr_type(node.condition, scope)
            if condition_type != LanguageType.BOOLEAN:
                raise TypeCheckError(self._error_at(node.condition, "While condition must have type Boolean"))
            self._check_block(node.body, scope.child_scope(), function_context)
            return

        if isinstance(node, Block):
            self._check_block(node, scope.child_scope(), function_context)
            return

        if isinstance(node, FunctionCall):
            self._infer_function_call_type(node, scope)
            return

        raise TypeCheckError(self._error_at(node, f"Unsupported statement node: {type(node).__name__}"))

    def _check_block(
        self,
        block: Block,
        scope: SymbolTable,
        function_context: _FunctionContext | None,
    ) -> None:
        for statement in block.statements:
            self._check_statement(statement, scope, function_context)

    def _infer_expr_type(self, node: ASTNode, scope: SymbolTable) -> LanguageType:
        if isinstance(node, Literal):
            return self._literal_type(node)

        if isinstance(node, Identifier):
            try:
                symbol = scope.lookup(node.name)
```

```python
            except SymbolTableError as error:
                raise TypeCheckError(self._error_at(node, str(error))) from error
            if not isinstance(symbol, VariableSymbol):
                raise TypeCheckError(self._error_at(node, f"'{node.name}' is not a variable"))
            return symbol.symbol_type

        if isinstance(node, FunctionCall):
            return self._infer_function_call_type(node, scope)

        if isinstance(node, BinaryOp):
            left_type = self._infer_expr_type(node.left, scope)
            right_type = self._infer_expr_type(node.right, scope)
            return self._infer_binary_type(node, left_type, right_type)

        raise TypeCheckError(self._error_at(node, f"Unsupported expression node: {type(node).__name__}"))

    def _infer_function_call_type(self, node: FunctionCall, scope: SymbolTable) -> LanguageType:
        function_state = self._functions.get(node.name)
        if function_state is None:
            raise TypeCheckError(self._error_at(node, f"Undefined function: {node.name}"))
        if node.name in self._active_calls:
            raise TypeCheckError(self._error_at(node, f"Recursive function calls are not supported: {node.name}"))

        argument_types = [self._infer_expr_type(argument, scope) for argument in node.args]
        function_symbol = self._lookup_function_symbol(node)

        if len(argument_types) != len(function_symbol.parameters):
            raise TypeCheckError(
                self._error_at(
                    node,
                    f"Function '{node.name}' expects {len(function_symbol.parameters)} argument(s), got {len(argument_types)}",
                )
            )

        declared_parameter_types = [param_type for _, param_type in function_symbol.parameters]
        if all(param_type == LanguageType.VOID for param_type in declared_parameter_types):
            function_symbol.parameters = list(zip(function_state.node.params, argument_types, strict=False))
        else:
            for index, ((_, expected_type), actual_type) in enumerate(zip(function_symbol.parameters, argument_types, strict=False)):
```

```
                if expected_type != actual_type:
                    raise TypeCheckError(
                        self._error_at(
                            node.args[index],
                            f"Argument {index + 1} of '{node.name}' must be {expected_type.value}, got {actual_type.value}",
                        )
                    )

        if not function_state.body_checked:
            self._check_function_body(function_state, function_symbol)

        return function_symbol.symbol_type

    def _check_function_body(self, function_state: _FunctionState, function_symbol: FunctionSymbol) -> None:
        function_context = _FunctionContext(name=function_state.node.name)
        function_scope = self.global_scope.child_scope()
        for parameter_name, parameter_type in function_symbol.parameters:
            function_scope.define_variable(parameter_name, parameter_type, initialized=True)

        self._active_calls.append(function_state.node.name)
        try:
            self._check_block(function_state.node.body, function_scope, function_context)
        finally:
            self._active_calls.pop()

        function_symbol.symbol_type = function_context.return_type or LanguageType.VOID
        function_state.body_checked = True

    def _check_uncalled_functions(self) -> None:
        """After all call-site checks, type-check any function whose body was never visited."""
        for name, state in self._functions.items():
            if not state.body_checked:
                try:
                    function_symbol = self.global_scope.lookup(name)
                except SymbolTableError:
                    continue
                if not isinstance(function_symbol, FunctionSymbol):
                    continue
                inferred_params = self._infer_param_types_from_body(state.node)
                function_symbol.parameters = inferred_params
                self._check_function_body(state, function_symbol)

    def _infer_param_types_from_body(self, node: FunctionDef) -> list[
```

```
tuple[str, LanguageType]]:
        """Infer parameter types by scanning the body for expression contexts.

        Any parameter used as an operand of an arithmetic or comparison operator
        is inferred as numeric (Float if the sibling operand is a float literal,
        Integer otherwise).  Parameters whose usage gives no type hint default to
        Integer.
        """
        param_names = set(node.params)
        inferred: dict[str, LanguageType] = {}

        def scan_expr(n: ASTNode) -> None:
            if isinstance(n, BinaryOp):
                if n.op in {"+", "-", "*", "/", "==", "!="}:
                    for operand, other in ((n.left, n.right), (n.right, n.left)):
                        if isinstance(operand, Identifier) and operand.name in param_names:
                            if isinstance(other, Literal) and isinstance(other.value, float):
                                inferred.setdefault(operand.name, LanguageType.FLOAT)
                            else:
                                inferred.setdefault(operand.name, LanguageType.INTEGER)
                scan_expr(n.left)
                scan_expr(n.right)
            elif isinstance(n, FunctionCall):
                for arg in n.args:
                    scan_expr(arg)

        def scan_stmt(n: ASTNode) -> None:
            if isinstance(n, Assign):
                scan_expr(n.value)
            elif isinstance(n, Print):
                scan_expr(n.expr)
            elif isinstance(n, Return):
                scan_expr(n.expr)
            elif isinstance(n, If):
                scan_expr(n.condition)
                scan_block(n.then_block)
                if n.else_block:
                    scan_block(n.else_block)
            elif isinstance(n, While):
                scan_expr(n.condition)
                scan_block(n.body)
            elif isinstance(n, Block):
                scan_block(n)
            elif isinstance(n, FunctionCall):
```

```
                for arg in n.args:
                    scan_expr(arg)

        def scan_block(block: Block) -> None:
            for stmt in block.statements:
                scan_stmt(stmt)

        scan_block(node.body)
        return [(param, inferred.get(param, LanguageType.INTEGER)) for param in node.params]

    def _assign_variable_type(self, scope: SymbolTable, node: Assign, value_type: LanguageType) -> None:
        try:
            symbol = scope.lookup(node.name)
        except SymbolTableError:
            scope.define_variable(node.name, value_type, initialized=True)
            return

        if not isinstance(symbol, VariableSymbol):
            raise TypeCheckError(self._error_at(node, f"'{node.name}' is not a variable"))
        if symbol.symbol_type != value_type:
            raise TypeCheckError(
                self._error_at(
                    node,
                    f"Cannot assign {value_type.value} to variable '{node.name}' of type {symbol.symbol_type.value}",
                )
            )
        symbol.initialized = True

    def _infer_binary_type(
        self,
        node: BinaryOp,
        left_type: LanguageType,
        right_type: LanguageType,
    ) -> LanguageType:
        if node.op in {"+", "-", "*", "/"}:
            if not self._is_numeric(left_type) or not self._is_numeric(right_type):
                raise TypeCheckError(self._error_at(node, f"Operator '{node.op}' requires numeric operands"))
            if node.op == "/":
                return LanguageType.FLOAT
            if left_type == LanguageType.FLOAT or right_type == LanguageType.FLOAT:
                return LanguageType.FLOAT
            return LanguageType.INTEGER

        if node.op in {"==", "!="}:
            numeric_comparison = self._is_numeric(left_type) and self.
```

```python
                _is_numeric(right_type)
            if not numeric_comparison:
                raise TypeCheckError(
                    self._error_at(
                        node,
                        f"Operator '{node.op}' requires two arithmetic operands",
                    )
                )
            return LanguageType.BOOLEAN

        raise TypeCheckError(self._error_at(node, f"Unsupported operator: {node.op}"))

    def _lookup_function_symbol(self, node: FunctionCall) -> FunctionSymbol:
        try:
            symbol = self.global_scope.lookup(node.name)
        except SymbolTableError as error:
            raise TypeCheckError(self._error_at(node, str(error))) from error
        if not isinstance(symbol, FunctionSymbol):
            raise TypeCheckError(self._error_at(node, f"'{node.name}' is not a function"))
        return symbol

    @staticmethod
    def _is_numeric(language_type: LanguageType) -> bool:
        return language_type in {LanguageType.INTEGER, LanguageType.FLOAT}

    @staticmethod
    def _literal_type(node: Literal) -> LanguageType:
        if isinstance(node.value, bool):
            return LanguageType.BOOLEAN
        if isinstance(node.value, int):
            return LanguageType.INTEGER
        if isinstance(node.value, float):
            return LanguageType.FLOAT
        if isinstance(node.value, str):
            return LanguageType.STRING
        raise TypeCheckError(f"Unsupported literal value: {node.value!r}")

    @staticmethod
    def _error_at(node: ASTNode, message: str) -> str:
        return f"[line {node.line}, col {node.column}] TypeError: {message}"
```

Listing 8.8: components/type_checker.py — static type checker with inference

```python
from __future__ import annotations

```

```python
from dataclasses import dataclass
from typing import Callable

from components.ast_nodes import (
    ASTNode,
    Assign,
    BinaryOp,
    Block,
    FunctionCall,
    FunctionDef,
    Identifier,
    If,
    Literal,
    Print,
    Program,
    Return,
    While,
)
from components.symbol_table import LanguageType


class InterpreterError(Exception):
    pass


class _ReturnSignal(Exception):
    def __init__(self, value: object) -> None:
        self.value = value


@dataclass
class _FunctionRuntime:
    node: FunctionDef


class _Environment:
    def __init__(self, parent: _Environment | None = None) -> None:
        self.parent = parent
        self.values: dict[str, object] = {}

    def define(self, name: str, value: object) -> None:
        self.values[name] = value

    def get(self, name: str) -> object:
        if name in self.values:
            return self.values[name]
        if self.parent is not None:
            return self.parent.get(name)
        raise InterpreterError(f"Undefined variable: {name}")

    def assign(self, name: str, value: object) -> None:
        if name in self.values:
            self.values[name] = value
```

```
            return
        if self.parent is not None and self.parent.contains(name):
            self.parent.assign(name, value)
            return
        self.values[name] = value

    def contains(self, name: str) -> bool:
        if name in self.values:
            return True
        if self.parent is not None:
            return self.parent.contains(name)
        return False


class Interpreter:
    def __init__(self, output_callback: Callable[[str], None] | None = None) -> None:
        self._functions: dict[str, _FunctionRuntime] = {}
        self._output_callback = output_callback or print

    def execute(self, program: Program) -> None:
        environment = _Environment()
        for statement in program.statements:
            if isinstance(statement, FunctionDef):
                self._functions[statement.name] = _FunctionRuntime(node=statement)

        for statement in program.statements:
            if isinstance(statement, FunctionDef):
                continue
            self._execute_statement(statement, environment)

    def _execute_statement(self, node: ASTNode, environment: _Environment) -> None:
        if isinstance(node, Assign):
            value = self._evaluate(node.value, environment)
            environment.assign(node.name, value)
            return

        if isinstance(node, Print):
            value = self._evaluate(node.expr, environment)
            self._output_callback(f"{self._format_value(value)} : {self._runtime_type(value).value}")
            return

        if isinstance(node, Return):
            raise _ReturnSignal(self._evaluate(node.expr, environment))

        if isinstance(node, If):
            condition = self._evaluate(node.condition, environment)
            if not isinstance(condition, bool):
                raise InterpreterError(self._error_at(node.condition, "If condition must evaluate to Boolean"))
```

```python
            branch = node.then_block if condition else node.else_block
            if branch is not None:
                self._execute_block(branch, _Environment(parent=environment))
            return

        if isinstance(node, While):
            while True:
                condition = self._evaluate(node.condition, environment)
                if not isinstance(condition, bool):
                    raise InterpreterError(self._error_at(node.condition, "While condition must evaluate to Boolean"))
                if not condition:
                    break
                self._execute_block(node.body, _Environment(parent=environment))
            return

        if isinstance(node, Block):
            self._execute_block(node, _Environment(parent=environment))
            return

        if isinstance(node, FunctionCall):
            self._evaluate(node, environment)
            return

        raise InterpreterError(self._error_at(node, f"Unsupported statement node: {type(node).__name__}"))

    def _execute_block(self, block: Block, environment: _Environment) -> None:
        for statement in block.statements:
            self._execute_statement(statement, environment)

    def _evaluate(self, node: ASTNode, environment: _Environment) -> object:
        if isinstance(node, Literal):
            return node.value

        if isinstance(node, Identifier):
            return environment.get(node.name)

        if isinstance(node, BinaryOp):
            left_value = self._evaluate(node.left, environment)
            right_value = self._evaluate(node.right, environment)
            return self._evaluate_binary(node, left_value, right_value)

        if isinstance(node, FunctionCall):
            return self._call_function(node, environment)

        raise InterpreterError(self._error_at(node, f"Unsupported expression node: {type(node).__name__}"))
```

```python

    def _call_function(self, node: FunctionCall, environment: _Environment) -> object:
        function_runtime = self._functions.get(node.name)
        if function_runtime is None:
            raise InterpreterError(self._error_at(node, f"Undefined function: {node.name}"))

        if len(node.args) != len(function_runtime.node.params):
            raise InterpreterError(
                self._error_at(
                    node,
                    f"Function '{node.name}' expects {len(function_runtime.node.params)} argument(s), got {len(node.args)}",
                )
            )

        call_environment = _Environment(parent=environment)
        argument_values = [self._evaluate(argument, environment) for argument in node.args]
        for parameter_name, argument_value in zip(function_runtime.node.params, argument_values, strict=False):
            call_environment.define(parameter_name, argument_value)

        try:
            self._execute_block(function_runtime.node.body, call_environment)
        except _ReturnSignal as signal:
            return signal.value
        return None

    def _evaluate_binary(self, node: BinaryOp, left_value: object, right_value: object) -> object:
        if node.op == "+":
            return left_value + right_value
        if node.op == "-":
            return left_value - right_value
        if node.op == "*":
            return left_value * right_value
        if node.op == "/":
            return left_value / right_value
        if node.op == "==":
            return left_value == right_value
        if node.op == "!=":
            return left_value != right_value
        raise InterpreterError(self._error_at(node, f"Unsupported operator: {node.op}"))

    @staticmethod
    def _format_value(value: object) -> str:
        if isinstance(value, str):
            return f'"{value}"'
```

```
        if isinstance(value, bool):
            return "true" if value else "false"
        return str(value)

    @staticmethod
    def _runtime_type(value: object) -> LanguageType:
        if isinstance(value, bool):
            return LanguageType.BOOLEAN
        if isinstance(value, int):
            return LanguageType.INTEGER
        if isinstance(value, float):
            return LanguageType.FLOAT
        if isinstance(value, str):
            return LanguageType.STRING
        return LanguageType.VOID

    @staticmethod
    def _error_at(node: ASTNode, message: str) -> str:
        return f"[line {node.line}, col {node.column}] RuntimeError: {
            message}"
```

Listing 8.9: components/interpreter.py — tree-walking interpreter

```
from __future__ import annotations

from dataclasses import dataclass

from components.ast_printer import ASTPrinter
from components.interpreter import Interpreter
from components.lexica import Lexer
from components.parser import Parser
from components.symbol_table import SymbolTable
from components.tokens import Token
from components.type_checker import TypeChecker


@dataclass
class PipelineResult:
    tokens: list[Token]
    ast_text: str
    checked_scope: SymbolTable
    outputs: list[str]


def run_pipeline(source_code: str) -> PipelineResult:
    tokens = Lexer(source_code).tokenize()
    tree = Parser(tokens).parse()
    ast_text = ASTPrinter().print(tree)
    checked_scope = TypeChecker().check(tree)

    outputs: list[str] = []
    Interpreter(output_callback=outputs.append).execute(tree)

```

```
    return PipelineResult(
        tokens=tokens,
        ast_text=ast_text,
        checked_scope=checked_scope,
        outputs=outputs,
    )


def format_tokens(tokens: list[Token]) -> str:
    lines = []
    for token in tokens:
        lines.append(
            f"  {token.token_type.name:<14} {token.lexeme!r:<12} line={token.line} col={token.column}"
        )
    return "\n".join(lines)


def format_stage_output(result: PipelineResult) -> str:
    sections = [
        "=" * 60,
        "  STAGE 1 -- TOKENS",
        "=" * 60,
        format_tokens(result.tokens),
        "",
        "=" * 60,
        "  STAGE 2 -- AST (parser output)",
        "=" * 60,
        result.ast_text,
        "",
        "=" * 60,
        "  STAGE 3 -- TYPE CHECK",
        "=" * 60,
        result.checked_scope.format_table(),
        "",
        "=" * 60,
        "  STAGE 4 -- EXECUTION",
        "=" * 60,
        "\n".join(result.outputs) if result.outputs else "(no output)",
    ]
    return "\n".join(sections)
```

Listing 8.10: components/pipeline.py — shared pipeline (CLI, UI, and tests)

A.3 Test Suite

```
from __future__ import annotations

import unittest

from components.lexica import Lexer, LexerError
from components.parser import Parser, ParseError
```

```python
from components.pipeline import run_pipeline
from components.symbol_table import LanguageType, SymbolTable, SymbolTableError
from components.tokens import TokenType
from components.type_checker import TypeCheckError


# ---------------------------------------------------------------------------
# Lexer
# ---------------------------------------------------------------------------

class LexerTests(unittest.TestCase):
    """Tests for components/lexica.py -- lexical analysis and tokenisation."""

    def test_integer_literal_token(self) -> None:
        tokens = Lexer("42").tokenize()
        self.assertEqual(tokens[0].token_type, TokenType.INT_LITERAL)
        self.assertEqual(tokens[0].lexeme, "42")

    def test_float_literal_token(self) -> None:
        tokens = Lexer("3.14").tokenize()
        self.assertEqual(tokens[0].token_type, TokenType.FLOAT_LITERAL)
        self.assertEqual(tokens[0].lexeme, "3.14")

    def test_bool_literals_produce_bool_literal_tokens(self) -> None:
        tokens = Lexer("true false").tokenize()
        self.assertEqual(tokens[0].token_type, TokenType.BOOL_LITERAL)
        self.assertEqual(tokens[0].lexeme, "true")
        self.assertEqual(tokens[1].token_type, TokenType.BOOL_LITERAL)
        self.assertEqual(tokens[1].lexeme, "false")

    def test_keywords_produce_correct_token_types(self) -> None:
        tokens = Lexer("if else while def return print").tokenize()
        expected = [
            TokenType.IF, TokenType.ELSE, TokenType.WHILE,
            TokenType.DEF, TokenType.RETURN, TokenType.PRINT,
        ]
        actual = [t.token_type for t in tokens if t.token_type != TokenType.EOF]
        self.assertEqual(actual, expected)

    def test_identifier_not_confused_with_keyword(self) -> None:
        tokens = Lexer("iffy whileloop").tokenize()
        self.assertEqual(tokens[0].token_type, TokenType.IDENTIFIER)
        self.assertEqual(tokens[1].token_type, TokenType.IDENTIFIER)

    def test_two_character_operators(self) -> None:
        tokens = Lexer("== !=").tokenize()
```

```
        self.assertEqual(tokens[0].token_type, TokenType.EQUAL_EQUAL)
        self.assertEqual(tokens[1].token_type, TokenType.BANG_EQUAL)

    def test_source_position_tracked_across_lines(self) -> None:
        tokens = Lexer("x\ny").tokenize()
        self.assertEqual(tokens[0].line, 1)
        self.assertEqual(tokens[1].line, 2)

    def test_unexpected_character_raises_lexer_error(self) -> None:
        with self.assertRaises(LexerError):
            Lexer("@").tokenize()

    def test_lone_exclamation_raises_lexer_error(self) -> None:
        with self.assertRaises(LexerError):
            Lexer("!5").tokenize()

    def test_unterminated_string_raises_lexer_error(self) -> None:
        with self.assertRaises(LexerError):
            Lexer('"hello').tokenize()


# ---------------------------------------------------------------------------
# Symbol table
# ---------------------------------------------------------------------------

class SymbolTableTests(unittest.TestCase):
    """Tests for components/symbol_table.py -- scoped symbol table and semantic types."""

    def test_define_variable_and_lookup(self) -> None:
        table = SymbolTable()
        table.define_variable("x", LanguageType.INTEGER, initialized=True)
        symbol = table.lookup("x")
        self.assertEqual(symbol.symbol_type, LanguageType.INTEGER)

    def test_duplicate_variable_definition_raises_error(self) -> None:
        table = SymbolTable()
        table.define_variable("x", LanguageType.INTEGER)
        with self.assertRaises(SymbolTableError):
            table.define_variable("x", LanguageType.FLOAT)

    def test_duplicate_function_definition_raises_error(self) -> None:
        table = SymbolTable()
        table.define_function("f", LanguageType.INTEGER, [])
        with self.assertRaises(SymbolTableError):
            table.define_function("f", LanguageType.FLOAT, [])

    def test_lookup_in_parent_scope(self) -> None:
```

```
        parent = SymbolTable()
        parent.define_variable("x", LanguageType.INTEGER, initialized=
            True)
        child = parent.child_scope()
        symbol = child.lookup("x")
        self.assertEqual(symbol.symbol_type, LanguageType.INTEGER)

    def test_undefined_symbol_raises_error(self) -> None:
        table = SymbolTable()
        with self.assertRaises(SymbolTableError):
            table.lookup("z")

    def test_format_table_contains_variable_row(self) -> None:
        table = SymbolTable()
        table.define_variable("counter", LanguageType.INTEGER,
            initialized=True)
        output = table.format_table()
        self.assertIn("counter", output)
        self.assertIn("variable", output)
        self.assertIn("Integer", output)

    def test_format_table_contains_function_row(self) -> None:
        table = SymbolTable()
        table.define_function("add", LanguageType.INTEGER, [("a",
            LanguageType.INTEGER)])
        output = table.format_table()
        self.assertIn("add", output)
        self.assertIn("function", output)


#
    ---------------------------------------------------------------------------
# Parser
#
    ---------------------------------------------------------------------------

class ParserTests(unittest.TestCase):
    """Tests for components/parser.py -- recursive-descent parsing."""

    def _parse(self, source: str):
        return Parser(Lexer(source).tokenize()).parse()

    def test_missing_semicolon_raises_parse_error(self) -> None:
        with self.assertRaises(ParseError):
            self._parse("x = 5")

    def test_unclosed_brace_raises_parse_error(self) -> None:
        with self.assertRaises(ParseError):
            self._parse("if (true) { x = 1;")

    def test_operator_precedence_mul_before_add(self) -> None:
```

```
        # 2 + 3 * 4 must be 14, not 20
        result = run_pipeline("x = 2 + 3 * 4; print(x);")
        self.assertEqual(result.outputs, ["14 : Integer"])

    def test_operator_left_associativity(self) -> None:
        # 10 - 3 - 2 must be 5 (left-assoc), not 9
        result = run_pipeline("x = 10 - 3 - 2; print(x);")
        self.assertEqual(result.outputs, ["5 : Integer"])

    def test_parentheses_override_precedence(self) -> None:
        # (2 + 3) * 4 must be 20
        result = run_pipeline("x = (2 + 3) * 4; print(x);")
        self.assertEqual(result.outputs, ["20 : Integer"])


# ---------------------------------------------------------------------------
# Type checker
# ---------------------------------------------------------------------------

class TypeCheckerTests(unittest.TestCase):
    """Tests for components/type_checker.py -- static type inference and checking."""

    def test_integer_arithmetic_stays_integer(self) -> None:
        result = run_pipeline("x = 3 + 4; print(x);")
        self.assertEqual(result.outputs, ["7 : Integer"])

    def test_division_always_produces_float(self) -> None:
        # integer / integer → Float by design
        result = run_pipeline("x = 10 / 2; print(x);")
        self.assertEqual(result.outputs, ["5.0 : Float"])

    def test_float_promotion_in_addition(self) -> None:
        result = run_pipeline("x = 1 + 2.5; print(x);")
        self.assertEqual(result.outputs, ["3.5 : Float"])

    def test_string_variable_type_inferred(self) -> None:
        result = run_pipeline('x = "hello"; print(x);')
        self.assertEqual(result.outputs, ['"hello" : String'])

    def test_boolean_literal_type_inferred(self) -> None:
        result = run_pipeline("x = true; print(x);")
        self.assertEqual(result.outputs, ["true : Boolean"])

    def test_boolean_equality_raises_type_error(self) -> None:
        # Boolean == Boolean is not allowed; == only accepts arithmetic operands
        with self.assertRaises(TypeCheckError):
```

```python
            run_pipeline('x = true; if (x == false) { print("then"); } else { print("else"); }')

    def test_arithmetic_on_string_raises_type_error(self) -> None:
        with self.assertRaises(TypeCheckError):
            run_pipeline('x = "a" + 1;')

    def test_if_non_boolean_condition_raises_type_error(self) -> None:
        with self.assertRaises(TypeCheckError):
            run_pipeline("if (1) { x = 2; }")

    def test_while_non_boolean_condition_raises_type_error(self) -> None:
        with self.assertRaises(TypeCheckError):
            run_pipeline("x = 0; while (x) { x = x + 1; }")

    def test_assignment_type_mismatch_raises_type_error(self) -> None:
        with self.assertRaises(TypeCheckError):
            run_pipeline('x = 5; x = "hello";')

    def test_undefined_variable_raises_type_error(self) -> None:
        with self.assertRaises(TypeCheckError):
            run_pipeline("print(z);")

    def test_function_wrong_arg_count_raises_type_error(self) -> None:
        with self.assertRaises(TypeCheckError):
            run_pipeline("def f(a) { return a; } f(1, 2);")

    def test_function_arg_type_mismatch_raises_type_error(self) -> None:
        # first call infers a: Integer; second call with String must fail
        with self.assertRaises(TypeCheckError):
            run_pipeline('def f(a) { return a; } f(1); f("hello");')

    def test_uncalled_function_body_type_error_is_caught(self) -> None:
        # type error inside body must be detected even if function is never called
        with self.assertRaises(TypeCheckError):
            run_pipeline('def bad() { y = "hello" + 1; }')

    def test_valid_uncalled_function_does_not_raise(self) -> None:
        # a well-typed but uncalled function should not raise
        result = run_pipeline("def add(a, b) { return a + b; }")
        self.assertEqual(result.outputs, [])


# ---------------------------------------------------------------------------
# Integration (full pipeline)
# ---------------------------------------------------------------------------
```

```python

class IntegrationTests(unittest.TestCase):
    """End-to-end tests through all four pipeline stages."""

    # --- Unary minus ---

    def test_unary_minus_integer(self) -> None:
        result = run_pipeline("x = -5; print(x);")
        self.assertEqual(result.outputs, ["-5 : Integer"])

    def test_unary_minus_float(self) -> None:
        result = run_pipeline("y = -3.14; print(y);")
        self.assertEqual(result.outputs, ["-3.14 : Float"])

    def test_unary_minus_variable(self) -> None:
        result = run_pipeline("x = 10; y = -x; print(y);")
        self.assertEqual(result.outputs, ["-10 : Integer"])

    def test_unary_minus_inside_expression(self) -> None:
        result = run_pipeline("x = 3 + -2; print(x);")
        self.assertEqual(result.outputs, ["1 : Integer"])

    # --- Control flow ---

    def test_if_then_branch_executes(self) -> None:
        result = run_pipeline(
            'x = 5; if (x == 5) { print("yes"); } else { print("no"); }'
        )
        self.assertEqual(result.outputs, ['"yes" : String'])

    def test_if_else_branch_executes_when_condition_false(self) -> None:
        result = run_pipeline(
            'x = 0; if (x == 5) { print("yes"); } else { print("no"); }'
        )
        self.assertEqual(result.outputs, ['"no" : String'])

    def test_while_loop_executes_repeatedly(self) -> None:
        result = run_pipeline("x = 3; while (x != 0) { print(x); x = x - 1; }")
        self.assertEqual(result.outputs, ["3 : Integer", "2 : Integer", "1 : Integer"])

    # --- Functions ---

    def test_function_integer_return(self) -> None:
        result = run_pipeline("def square(n) { return n * n; } print(square(7));")
        self.assertEqual(result.outputs, ["49 : Integer"])

```

```python
    def test_function_type_inference(self) -> None:
        result = run_pipeline(
            "def add(a, b) { return a + b; } result = add(2, 3.5); print(result);"
        )
        self.assertEqual(result.outputs, ["5.5 : Float"])
        self.assertIn("result       variable   Float", result.checked_scope.format_table())

    def test_function_value_parameter_passing(self) -> None:
        # Modifying a parameter inside a function must not affect the caller's variable
        result = run_pipeline(
            "def inc(a) { a = a + 1; return a; } x = 10; y = inc(x); print(x); print(y);"
        )
        self.assertEqual(result.outputs, ["10 : Integer", "11 : Integer"])

    def test_nested_function_calls(self) -> None:
        result = run_pipeline(
            "def add(a, b) { return a + b; } print(add(add(1, 2), 3));"
        )
        self.assertEqual(result.outputs, ["6 : Integer"])

    # --- print() with all four types ---

    def test_print_integer(self) -> None:
        result = run_pipeline("print(42);")
        self.assertEqual(result.outputs, ["42 : Integer"])

    def test_print_float(self) -> None:
        result = run_pipeline("print(3.14);")
        self.assertEqual(result.outputs, ["3.14 : Float"])

    def test_print_boolean(self) -> None:
        result = run_pipeline("print(true);")
        self.assertEqual(result.outputs, ["true : Boolean"])

    def test_print_string(self) -> None:
        result = run_pipeline('print("plc");')
        self.assertEqual(result.outputs, ['"plc" : String'])

    # --- Reassignment ---

    def test_variable_can_be_reassigned_same_type(self) -> None:
        result = run_pipeline("x = 1; x = 2; x = 3; print(x);")
        self.assertEqual(result.outputs, ["3 : Integer"])

    def test_scope_isolation_if_block(self) -> None:
        # Variable defined inside an if-block must not be visible outside it
        with self.assertRaises(TypeCheckError):
```

```python
            run_pipeline(
                "x = 1; if (x == 1) { inner = 99; } print(inner);"
            )


if __name__ == "__main__":
    unittest.main()
```

Listing 8.11: tests/test_pipeline.py — automated regression suite (54 tests)